

SLOVAK UNIVERSITY OF TECHNOLOGY IN BRATISLAVA
Faculty of Informatics and Information Technologies

Reg. No.: FIIT-182905-102924

**Application of computer vision methods using
reinforcement learning in the football domain.**

Master thesis

Study programme: Intelligent Software Systems

Study field: Computer Science

Training workplace: Institute of Computer Engineering and Applied Informatics

Thesis supervisor: Ing. Igor Jánoš, PhD.

Bratislava 2025

Bc. Roman Grom



MASTER THESIS TOPIC

Student: **Bc. Roman Grom**
Student's ID: 102924
Study programme: Intelligent Software Systems
Study field: Computer Science
Thesis supervisor: Ing. Igor Jánoš, PhD.
Head of department: Ing. Katarína Jelemenská, PhD.

Topic: **Application of computer vision methods using reinforcement learning in the football domain.**

Language of thesis: English

Specification of Assignment:

V športovej doméne prišlo v nedávnej dobe k veľkému nárastu vo využívaní nástrojov a pomôcok využívajúcich počítačové videnie. Jedná sa o štatistické systémy, systémy sledujúce pohyb hráčov, alebo video analytické pomôcky pre rozhodcov. Na celom svete sa dennodenne odohrá veľké množstvo futbalových zápasov, ktoré trvajú častokrát aj viac než stanovených 90 minút. V každom zápase sa ale udeje iba malé množstvo naozaj dôležitých udalostí. Pre urýchlenie práce so záznamami je potrebné porozumieť záznamu a vedieť v nich automatizovane identifikovať dôležité momenty. Niektoré sú ľahko rozoznateľné (gól, žltá karta), iné naopak nejasné – šanca, pričom práve šance bývajú pre divákov najatraktívnejšie. Analyzujte existujúce metódy identifikácie dôležitých momentov vo futbalových záznamoch s využitím umelých neurónových sietí. Analyzujte existujúce metódy učenia posilňovaním a jeho možnosti použitia pre identifikáciu dôležitých momentov. Zamerajte sa na metódy, ktoré by nevyžadovali učenie na anotovaných dátach, ale využívali by interakciu s prostredím simulujúcim futbal. Navrhnite vlastnú metódu identifikácie šanci vo videozázname futbalového zápasu s využitím agenta trénovaného posilňovaním v prostredí simulujúcom futbal. Implementujte Vami navrhnutú metódu s využitím dostupných knižníc a rámcov vhodných pre úlohy počítačového videnia. Metódu vyhodnot'ite na vlastnom datasete alebo inom vereje dostupnom datasete futbalových záznamov. Jej efektivitu, výhody a nevýhody porovnajte voči iným publikovaným riešeniam.

Deadline for submission of Master thesis: 11. 05. 2025
Approval of assignment of Master thesis: 15. 04. 2025
Assignment of Master thesis approved by: prof. Ing. Vanda Benešová, PhD. – Study programme supervisor

Čestne vyhlasujem, že som túto prácu vypracoval samostatne, na základe konzultácií a s použitím uvedenej literatúry.

Bratislava, 11.5.2025

.....

Roman Grom

Pod'akovanie

Ďakujem vedúcemu práce za korektný prístup a neustále usmerňovanie ma správnym smerom. Taktiež ďakujem kamarátom Tomášovi a Richardovi, bez ktorých by som veľa vecí nedokázal.

Anotácia

Slovenská technická univerzita v Bratislave

FAKULTA INFORMATIKY A INFORMAČNÝCH TECHNOLOGIÍ

Študijný program: inteligentné softvérové systémy

Autor: Roman Grom

Diplomová práca: Aplikácia metód počítačového videnia s použitím učenia posilňovaním v doméne obrazového záznamu futbalu

Vedúci diplomovej práce: Ing. Igor Jánoš, PhD.

Každý deň sa po celom svete odohrá veľké množstvo futbalových zápasov, pri ktorých vzniká veľké množstvo obrazových záznamov. Ich automatické spracovanie je jednou z výziev strojového učenia. Jednou z oblastí záujmu je automatické rozpoznávanie kľúčových momentov zápasu. Medzi ne patria aj príležitosti na skórovanie (šance), ktoré sú pre divákov zaujímavé. Šance sú špecifické tým, že nie sú presne definované, čo komplikuje prácu s nimi. V tejto práci skúmame dostupné metódy počítačového videnia a metódy učenia posilňovaním vhodné na rozpoznávanie šancí z obrazových záznamov futbalových zápasov. Nami navrhnuté riešenie využíva schopnosť agenta učenia posilňovaním (actor-critic metóda) učiť sa odhadovať hodnotu jednotlivých stavov hry. Vyššie hodnoty by mohli korelovať s výskytom šancí. Agentu učíme hrať futbal na simulovanom prostredí a následne prenášame jeho naučené schopnosti na reálne futbalové záznamy. Táto práca obsahuje najskôr analýzu využitých metód, a to metód neurónových sietí, počítačového videnia a učenia posilňovaním. Ďalej obsahuje prehľad existujúcich riešení. V nasledujúcej časti podrobne opisujeme nami navrhnuté riešenie. Na konci práce opisujeme naše experimenty a vyhodnocujeme úspešnosť a využiteľnosť nášho riešenia.

Annotation

Slovak University of Technology Bratislava

FACULTY OF INFORMATICS AND INFORMATION TECHNOLOGIES

Degree Course: intelligent software systems

Author: Roman Grom

Master's Thesis: Application of computer vision methods using reinforcement learning in the football domain

Supervisor: Ing. Igor Jánoš, PhD.

2025, may

Every day, numerous football matches occur globally, generating extensive video recordings. The automated processing of these recordings presents a significant challenge in machine learning. One area of interest is the automatic recognition of key match moments, particularly goal-scoring opportunities (chances), which are highly engaging for viewers. Chances are unique due to their lack of precise definition, complicating their analysis. This work explores available computer vision and reinforcement learning methods for identifying chances from football match videos. Our proposed solution leverages the capability of a reinforcement learning agent (actor-critic method) to learn and estimate the value of individual game states. They can achieve highly accurate results in various domains and often outperform traditional algorithms such as decision trees, logistic regression, or support vector machines. We train the agent to play football in a simulated environment and transfer its learned skills to real football footage. This paper begins with an analysis of the employed methods, including neural networks, computer vision, and reinforcement learning. It further provides an overview of existing solutions. The following section details our proposed solution. Finally,

x

we describe our experiments and evaluate the effectiveness and applicability of our approach.

Contents

Introduction	1
1 Analysis of Neural Network Methods	3
1.1 Neural Networks	3
1.2 Convolutional Neural Networks	5
1.3 Vision Transformer	7
1.4 Feature Extraction	9
2 Analysis of Reinforcement Learning Methods	11
2.1 Reinforcement Learning	11
2.1.1 Finite Markov Decision Processes	13
2.1.2 Dynamic Programming	14
2.1.3 Monte Carlo Methods	17
2.1.4 Temporal-Difference Learning	17
2.2 DQN	19
2.3 Actor-Critic Methods	20
2.4 Asynchronous Advantage Actor Critic	22
2.5 PPO	23
3 Related Work	25

3.1	SoccerNet-v2: A Dataset and Benchmarks for Holistic Understanding of Broadcast Soccer Videos	25
3.2	Automatic Highlights Generation for Sports Videos Using Audio-Visual Cues	26
3.3	Event Detection in Football using Graph Convolutional Networks .	27
3.4	Summary of Related Work	28
4	Our Approach	29
4.1	Introduction to Our Problem	29
4.2	Solution Design	30
4.3	Training Environment	32
4.3.1	Adapting the Environment	33
4.4	Agent Model Architecture	34
4.5	Training the Agent	36
4.6	Validation Dataset Preparation	37
4.6.1	Supplementary Manually-Annotated Dataset	38
4.7	Using Real-World Videos to Validate the Critic	39
4.7.1	Preparing Real-World Observations	40
4.7.2	Evaluation of the Critic	41
5	Experiments and Results	45
5.1	Evaluating the Critic on the Simulator	45
5.2	Classic Actor-Critic PPO Approach	47
5.3	Only-AI Critic Approach	49
5.4	Only-AI Critic with Lower Discount Factor	50
5.5	Special Cases	51
6	Conclusion	55

	xiii
Bibliography	57
Appendix A: Resumé v slovenčine	A-1
Appendix B: Opis digitálnej časti	B-1

Introduction

Football is the most popular and widely watched sport in the world. Every day, a vast number of matches are played globally, often exceeding the standard 90-minute duration. These matches generate a large volume of video recordings, the automated processing of which represents a significant challenge in modern machine learning. While matches are lengthy, the number of truly noteworthy events (key moments) is relatively small. Therefore, developing automated tools for their identification is highly advantageous. Such tools can eliminate the need for manual review of long recordings, for instance, when creating highlight reels of important moments.

Key moments, such as goals, penalties, or red cards, are relatively straightforward to detect and are clearly defined. However, key moments also include goal-scoring opportunities (chances), particularly major ones. The definition of what constitutes a chance varies among individuals and lacks a precise, universally accepted definition. This characteristic distinguishes chances from other key moments. Consequently, developing an automated tool for the recognition of chances would be highly beneficial, though it requires a specialized approach.

This thesis explores and analyzes methods suitable for developing a tool capable of automatically identifying goal-scoring opportunities in football video recordings. Our primary focus is on computer vision techniques combined with reinforcement learning. Our approach involves training a reinforcement learning

agent using the actor-critic method in a simulated football environment. In addition to playing football, the agent learns to numerically evaluate different game states. We then attempt to apply this trained agent to real football videos, expecting that higher numerical evaluations will correspond to goal-scoring opportunities.

The thesis consists of two main parts. The first part is analytical, providing a detailed explanation and analysis of the methods applicable to our problem. These methods primarily stem from the fields of neural networks, computer vision, and reinforcement learning. The discussion typically progresses from general approaches to specific techniques that we employ. A dedicated chapter examines existing solutions addressing similar problems. The second part describes our approach in detail, covering its design, implementation, and evaluation. Finally, the thesis presents the conducted experiments and assesses the effectiveness and applicability of our solution.

1. Analysis of Neural Network Methods

1.1 Neural Networks

Neural networks (or artificial neural networks) are among the most successful machine learning models. They can achieve highly accurate results in various domains and often outperform traditional algorithms. They are beneficial in computer vision and audio processing, where conventional approaches often fall behind [1].

Neural networks are inspired by the human brain. The brain consists of numerous neurons connected by synapses, while ANNs are composed of perceptrons organized into layers. These networks typically have an input layer, an output layer, and an arbitrary number of hidden layers in between. The input layer receives a vector of numerical values. As the data passes through the network, a series of transformations is performed (using weighted connections and activation functions that introduce non-linearity) until the output is produced by the output layer. This process is known as forward propagation [2].

Neural networks optimize their predictions for a given problem through a process called backpropagation (backward propagation of errors), in which errors are propagated backward through the network to adjust its parameters (weights and

biases) [2]. Optimization algorithms such as Stochastic Gradient Descent (SGD) or Adam iteratively update these parameters to minimize prediction errors. For example, in the case of SGD, the weight parameters w_t are updated as follows:

$$w_{t+1} := w_t - \alpha \frac{\partial \mathcal{L}}{\partial w_t}$$

where α is the learning rate, $\mathcal{L}$ is the loss function, and $\frac{\partial \mathcal{L}}{\partial w_t}$ is the gradient of the loss with respect to the weights at time step t . Training occurs over multiple epochs until the model reaches optimal or satisfactory performance. Simply put, a neural network is a highly complex mathematical function that dynamically adjusts its parameters using optimization algorithms to better fit the data [1].

Neural networks are trained on domain-specific datasets, which can be either labeled (supervised learning) or unlabeled (unsupervised learning). The goal of training is to minimize a loss function, which quantifies the difference between the network's prediction and the actual ground truth. Common loss functions include mean squared error (MSE) for regression tasks and cross-entropy loss for classification tasks [3].

Depending on the dataset, task type, and other constraints, different neural network architectures and algorithms exist. While this work does not cover all of them, some notable examples include Multilayer Perceptrons (MLPs)[2], which are fully connected networks primarily used for general-purpose machine learning tasks, Convolutional Neural Networks (CNNs)[4] for image processing, Recurrent Neural Networks (RNNs)[5] for sequential data, and transformer-based models[6] for natural language processing (NLP).

Neural networks aim to approximate functions within a given problem domain rather than perfectly replicate them. Achieving 100% accuracy is practically impossible due to challenges such as noise in data, problem complexity, and real-world variability. Additionally, training can suffer from overfitting, where the

model memorizes training data instead of generalizing to new inputs (a common issue mitigated by regularization techniques such as dropout). Neural networks are particularly valuable in problems where the target function is unknown or computationally too expensive to derive [7].

1.2 Convolutional Neural Networks

Artificial intelligence (AI) aims to achieve human capabilities and, in some areas, even surpass them. One of the most successful fields of AI is computer vision, which enables computers to "see" and interpret visual data from the real world. A key type of neural network for computer vision is convolutional neural networks.

A Convolutional Neural Network (ConvNet) is a type of deep learning model designed to process input images. It can identify various features within the images, assign importance to these features using learnable weights and biases, and ultimately recognize what is depicted in the image or perform other processing tasks [8].

One advantage of ConvNets, compared to traditional machine learning methods, is that they can automatically learn relevant features from raw data, reducing the need for extensive manual feature extraction. In conventional techniques, filters are often created manually; however, ConvNets can learn to generate these filters and other essential parameters during the training process [8].

As input for ConvNets, a raster image is typically used, meaning an image represented by pixels. Most commonly, it consists of three channels (RGB space), as shown in the figure 1.1

$$C(i, j) = (I * K)(i, j) = \sum_m^M \sum_n^N I(i - m, j - n) K(m, n) \quad (1.1)$$

The convolution operation between an image matrix and a filter matrix is

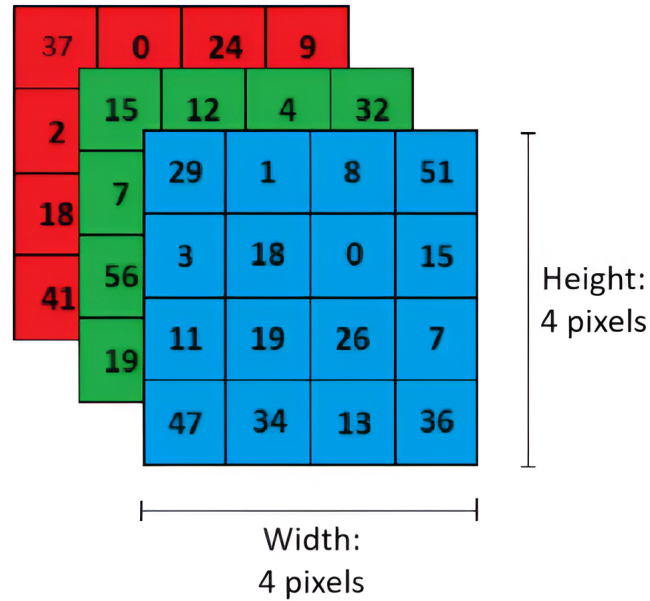


Figure 1.1: A typical structure of an input image, with three color channels.

formally defined by equation 1.1. This process involves iteratively calculating the dot products of all values from the image matrix with those from the inverted filter matrix. Additionally, there is a cross-correlation function that is similar to convolution but does not invert the filters [1].

In ConvNet architecture, multiple convolutional layers are typically used. Lower layers learn to recognize simple shapes such as lines, edges, colors, or orientations. Higher layers are capable of recognizing more complex patterns. The final layers should be able to understand relationships between simpler features and identify more complex objects in a manner similar to human perception [8].

Pooling layers are used between convolutional layers. They are responsible for reducing the size of the image as it passes through the network while preserving important information. This reduction in size helps decrease computational complexity and increases the receptive field. There are two commonly used types of

pooling layers: Max pooling and Average pooling. Max Pooling selects the maximum value from the region covered by the filter, while Average Pooling computes the average value over that same area.[8].

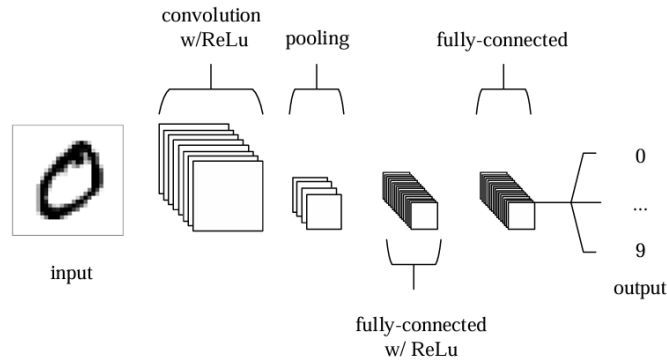


Figure 1.2: Architecture of convolutional neural network [8].

Typically, one or more fully connected layers are added after the last convolutional layer (Figure 1.2). They take its flattened output and can efficiently learn nonlinear combinations of high-level features. In classification tasks, the final fully connected layer often uses a softmax activation function to produce class probabilities, while in regression tasks, a linear activation function is commonly applied. During training, the backward propagation of errors is used, similar to MLPs, but instead of adjusting weights in fully connected layers, the filters and biases of the convolutional layers are optimized [8].

1.3 Vision Transformer

The Vision Transformer (ViT), which leverages the self-attention mechanism of the Transformer architecture, has achieved significant success in recent years. In natural language processing, Transformers have become the dominant model of choice. Thanks to the self-attention mechanism, they can efficiently process

sequential data, analyze words, and weigh their relationships in the context of an entire sentence [9].

Inspired by their success, various studies have attempted to combine ConvNets with the self-attention mechanism. However, the most successful approach has been to apply a standard Transformer directly to images. Unlike ConvNets, the self-attention mechanism in Transformers can capture global relationships in images from the first layers.

Such a model is known as the Vision Transformer (ViT). A Vision Transformer processes images by dividing them into patches, flattening them into vectors, and projecting them into an embedding space of the required dimension using a linear layer. Positional embeddings are then added to these embeddings to preserve spatial information. Image patches are treated in the same way as word tokens in NLP. Its architecture is shown in the figure 1.3

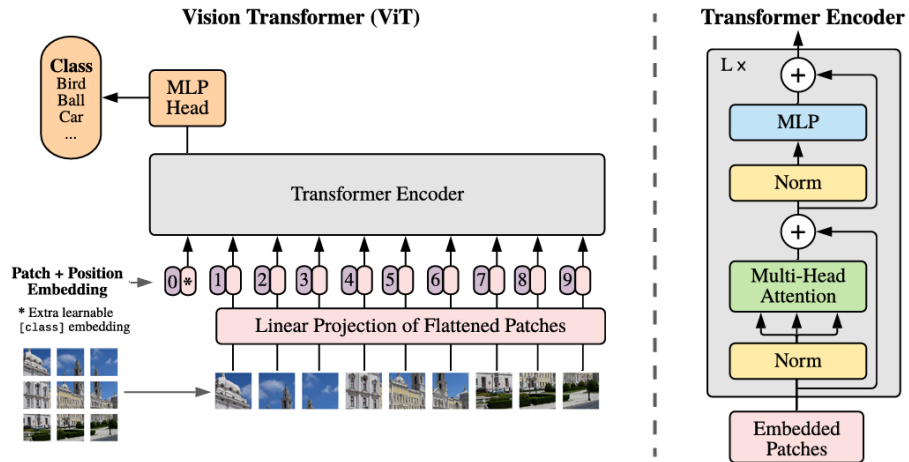


Figure 1.3: Architecture of a Vision Transformer [9].

Vision Transformers (ViTs) are commonly trained in a supervised fashion for image classification tasks. Their performance advantage over traditional convolutional networks becomes particularly noticeable when scaled to large datasets. When pre-trained on datasets like the publicly available ImageNet-21k or Google's

internal JFT-300M, ViTs match or outperform state-of-the-art methods across several image recognition benchmarks. Specifically, the top-performing model reaches 88.55% accuracy on ImageNet, 90.72% on ImageNet-ReaL, 94.55% on CIFAR-100, and 77.63% on the 19-task VTAB benchmark suite. [9].

1.4 Feature Extraction

Feature extraction is a technique that transforms raw data, commonly in the form of images, into numerical features. This process allows for further analysis while preserving the original information. Compared to directly applying machine learning to raw data, feature extraction typically leads to improved performance. [10].

Elements can be extracted either manually or automatically. Manual feature extraction involves identifying and describing relevant characteristics for a specific situation, followed by implementing the extraction method. While having a solid understanding of the problem domain is beneficial, automated feature extraction is often preferred. This approach uses specialized algorithms or deep learning networks to automatically extract information from signals or images without human intervention. This method can be particularly useful when there is a need to quickly transition from collecting raw data to building machine learning algorithms [10].

With the rise of deep learning, traditional feature extraction methods have largely been replaced by the initial layers of deep convolutional networks, particularly in the context of image data. Feature extraction for images involves representing the significant parts of an image as a compact feature vector. Previously, specialized techniques for feature identification, extraction, and matching were commonly used.

However, deep learning has become the predominant approach for image and

video analysis, renowned for its ability to analyze raw image data without the need for domain expertise and prior feature extraction.

Regardless of the method used, computer vision applications such as image registration, object detection and classification, and content-based image retrieval require an efficient representation of image features. This representation can be achieved either implicitly through the first layers of a deep network or explicitly using established image feature extraction techniques [10].

2. Analysis of Reinforcement Learning Methods

2.1 Reinforcement Learning

Reinforcement Learning (RL) is a field of machine learning that attempts to replicate the way humans learn to make decisions. The main goal of RL is to figure out which actions to take in different situations in order to maximize a numerical reward. In this process, the learner, known as the agent, is required to explore various actions and determine which ones lead to the best rewards. Unlike some other learning methods, the agent is not directly told what to do but must discover the best actions through experience [11].

In more complex scenarios, the agent's actions can affect both immediate and future rewards. Two key features that make reinforcement learning unique are exploration through trial and error and the fact that rewards can be delayed [11].

Reinforcement learning differs from both supervised and unsupervised learning. In supervised learning, a teacher provides labeled examples for the algorithm to learn from. In contrast, unsupervised learning deals with unlabeled data, seeking hidden patterns within it. Therefore, reinforcement learning is a distinct category of machine learning, separate from the other two [11].

In RL, there are two main components: the agent and the environment. The

agent is the entity that interacts with the environment by making decisions and performing actions, while everything else is considered the environment. Even though the agent faces uncertainty in its surroundings, it strives to reach a specific goal. As the agent interacts more with the environment, its performance improves [11].

In addition to the agent and the environment, a reinforcement learning system consists of four primary elements: a policy, a reward signal, a value function, and, when present, a model of the environment. The policy defines how the agent should behave in a given situation. It essentially maps what the agent perceives from the environment to the actions it should take. This is similar to a rule or stimulus-response relationship in psychology. The policy could be as simple as a function or a table, or it could involve more complex algorithms, depending on the situation. The policy is a core component of the agent, as it alone can control its behavior — the actions. The policy can typically be stochastic [11].

Another component is the reward signal. The environment sends a numerical value known as the reward to the agent at each time step (after every action). This reward signal indicates how good the action taken by the agent was. The goal of the agent is to maximize its total return in the long run. Rewards can be thought of as analogous to feelings of pleasure or pain in biological systems. The reward signal serves as the primary reason for policy change. If the action chosen by the policy results in a low reward, the policy might be adjusted to choose a different action in similar future circumstances. Reward signals are typically stochastic functions of the environment's state and the actions taken [11].

As the final component, an agent may or may not include a model of the environment. This model mimics the environment's behavior or, more generally, enables the prediction of future environmental states. For example, a model could predict the subsequent state and reward given a particular state and action. Models

are used for planning, which we define as any method of selecting a course of action by considering potential future outcomes before they occur. Model-based approaches to reinforcement learning problems differ from simpler model-free approaches that strictly rely on trial-and-error learning, which is considered the opposite of planning, where models and planning are used [11].

2.1.1 Finite Markov Decision Processes

Finite Markov Decision Processes (MDPs) provide a formal framework for understanding the key concepts and operations behind Reinforcement Learning. The agent acts as both the learner and decision-maker. The environment, which includes everything outside the agent, is what the agent interacts with. The agent chooses actions, and the environment responds by presenting new states to the agent, as depicted in Figure 2.1. In addition, the environment generates rewards, which are numerical values that the agent aims to maximize by making appropriate decisions over time [11].

At each discrete time step $t = 0, 1, 2, \dots$, the agent interacts with the environment by observing a state representation $S_t \in S$, and selecting an action $A_t \in A(S_t)$. Following the action, the agent transitions to a new state S_{t+1} and receives a scalar reward $R_{t+1} \in R \subset \mathbb{R}$ [11].

In the case of a finite MDP, the sets of possible states S , actions A , and rewards R are all finite. The random variables R_t and S_t follow discrete probability distributions that depend solely on the preceding state and action [11]. Specifically, for certain values of these random variables, $s' \in S$ and $r \in R$, there is a probability of these values occurring at time t , given the previous state and action:

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a\}, \quad (2.1)$$

for all $s', s \in S$, $r \in R$, and $a \in A(s)$. For any choice of $s \in S$ and $a \in A$, the sum of the probabilities of all possible $s' \in S$ and $r \in R$ must satisfy the following equation:

$$\sum_{s' \in S} \sum_{r \in R} p(s', r | s, a) = 1.$$

The probability function p fully characterizes a finite MDP. With this information, we can compute various other elements of the environment, such as the state-transition probabilities [11].

$$p(s' | s, a) \doteq \Pr\{S_t = s' | S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in R} p(s', r | s, a). \quad (2.2)$$

We can compute the expected rewards for state-action pairs as a function $r : S \times A \rightarrow R$:

$$r(s, a) \doteq E[R_t | S_{t-1} = s, A_{t-1} = a] = \sum_{r \in R} r \sum_{s' \in S} p(s', r | s, a), \quad (2.3)$$

or the expected rewards for state-action-next-state triples as a three-argument function $r : S \times A \times S \rightarrow R$,

$$r(s, a, s') \doteq E[R_t | S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in R} r \frac{p(s', r | s, a)}{p(s' | s, a)}. \quad (2.4)$$

2.1.2 Dynamic Programming

Dynamic programming (DP) refers to a set of methods used to find the optimal policy when the environment is accurately modeled as a Markov Decision Process

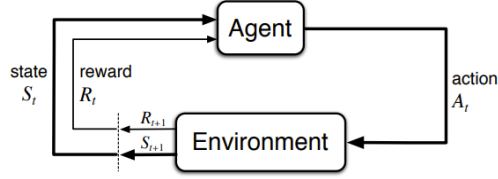


Figure 2.1: The agent-environment interaction [11].

(MDP). The use of value functions to guide the search for effective policies is a core concept in DP and Reinforcement Learning. Optimal policies can be determined once the optimal value functions— $v_*(s)$, the optimal state-value function, and $q_*(s, a)$, the optimal action-value function—are found, as these satisfy the Bellman optimality equations:

$$\begin{aligned}
 v_*(s) &= \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) | A_t = a] \\
 &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]
 \end{aligned} \tag{2.5}$$

$$\begin{aligned}
 q_*(s, a) &= E[R_{t+1} + \gamma \max_a q_*(S_{t+1}, a') | A_t = a] \\
 &= \sum_{s', r} p(s', r | s, a) [r + \gamma \max_a q_*(s', a')]
 \end{aligned} \tag{2.6}$$

for all $s \in S$, $a \in A(s)$, and $s' \in S^+$. Here, $\gamma \in [0, 1]$ is the discount factor for future rewards. DP algorithms are developed by transforming the Bellman equations into practical update rules, which iteratively refine approximations of the target value functions [11].

Consider a sequence of approximate value functions $v_0, v_1, v_2, \dots$, each mapping S^+ to $\mathbb{R}$. The initial approximation, v_0 , is chosen arbitrarily, and each subsequent approximation is derived by using the Bellman equation for v_π as an

update rule:

$$\begin{aligned}
 v_{k+1}(s) &= E[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s] \\
 &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]
 \end{aligned} \tag{2.7}$$

for all $s \in S$. This algorithm is known as iterative policy evaluation [11].

The primary goal of computing the value function v_π for a given policy π is to facilitate the discovery of improved policies. Suppose we have already evaluated v_π for a particular deterministic policy. For a given state s , we may want to assess whether it would be beneficial to alter the policy by deterministically selecting an action $a \neq \pi(s)$. To determine whether such a change would lead to better outcomes, we consider taking action a in state s , and then continuing to follow the original policy π . The value of this new behavior is:

$$\begin{aligned}
 q_\pi(s, a) &= E[R_{t+1} + \gamma v_\pi(S_{t+1}, a') | A_t = a] \\
 &= \sum_{s', r} p(s', r | s, a) [r + \gamma q_\pi(s')].
 \end{aligned} \tag{2.8}$$

The key criterion is whether the expected return from the new action is greater than or less than $v_\pi(s)$. The process of generating a new policy by acting greedily with respect to the value function is referred to as *policy improvement*. After improving a policy π using its corresponding value function v_π , we obtain a better policy π' . We can then evaluate $v_{\pi'}$ and repeat the improvement step to derive an even better policy π'' . This iterative process of alternately evaluating and improving policies produces a sequence of monotonically improving policies and associated value functions. This approach is known as *policy iteration* [11].

2.1.3 Monte Carlo Methods

Reinforcement Learning (RL) problems can be tackled using Monte Carlo (MC) methods, which assume that the learning process is divided into episodes that eventually terminate, regardless of the actions taken within them. In MC methods, both the value estimates and the policy are updated only at the conclusion of each episode. Consequently, learning occurs on an episode-by-episode basis rather than at each time step. For each state-action pair encountered, MC algorithms estimate the value by sampling and averaging the observed returns from those visits [11].

The value of a state is defined as the expected return—the expected cumulative discounted reward starting from that state onward. A straightforward and effective way to estimate this expectation from empirical data is by averaging the actual returns received during episodes where the state was visited. Given enough samples, this average converges to the true expected return. This fundamental principle underpins all Monte Carlo methods [11].

Monte Carlo methods can also be applied to control problems to approximate optimal policies. The general approach is similar to dynamic programming and follows the framework of *generalized policy iteration* (GPI). In this framework, both the policy and the value function are approximated and updated iteratively: the policy is adjusted continuously based on the current value function, and the value function is periodically refined to better approximate the value of the updated policy [11].

2.1.4 Temporal-Difference Learning

Dynamic programming (DP) encompasses a set of methods designed to determine the optimal policy when the environment is fully represented as a Markov decision process. One of the key concepts in both DP and Reinforcement Learning is the

use of value functions to guide the search for effective policies. Once the optimal value functions— $v_*(s)$, the optimal state-value function, and $q_*(s, a)$, the optimal action-value function—are found, optimal policies can be derived. These value functions must satisfy the Bellman optimality equations:

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \end{aligned} \quad (2.9)$$

$$\begin{aligned} q_*(s, a) &= \mathbb{E}[R_{t+1} + \gamma \max_a q_*(S_{t+1}, a') | A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) [r + \gamma \max_a q_*(s', a')] \end{aligned} \quad (2.10)$$

for all $s \in S$, $a \in A(s)$, and $s' \in S^+$. The discount factor $\gamma \in [0, 1]$ determines the importance of future rewards. DP algorithms are developed by transforming the Bellman equations into practical update rules that iteratively refine approximations of these target value functions [11].

Consider a series of approximated value functions $v_0, v_1, v_2, \dots$, each mapping S^+ to R . The initial approximation, v_0 , is chosen arbitrarily, and each subsequent approximation is derived by applying the Bellman equation for v_π as an update rule:

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')] \end{aligned} \quad (2.11)$$

for all $s \in S$. This method is known as iterative policy evaluation [11].

The goal of computing the value function v_π for a given policy π is to facilitate the discovery of improved policies. Suppose we have already evaluated v_π for a deterministic policy. In this case, we may want to determine whether altering the policy at a given state s , such that it deterministically chooses an action $a \neq \pi(s)$, would lead to better results. To assess the potential benefit, we evaluate the value of taking action a at state s , and then following the original policy π thereafter:

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}, a') | A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) [r + \gamma q_\pi(s')]. \end{aligned} \quad (2.12)$$

The key criterion is whether the expected return from the new action exceeds $v_\pi(s)$. The process of creating a new policy by acting greedily with respect to its value function is called *policy improvement*. After improving the policy π using its value function v_π , we obtain a better policy π' . We can then evaluate $v_{\pi'}$ and repeat the process to obtain an even better policy π'' . This cycle of policy evaluation and improvement results in a sequence of policies and value functions that monotonically improve. This method is known as *policy iteration* [11].

2.2 DQN

The development of Q-learning, an off-policy temporal difference (TD) control algorithm, marked a major breakthrough in Reinforcement Learning [12]. The simplest form of Q-learning, known as one-step Q-learning, follows this update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]. \quad (2.13)$$

This method allows the learned action-value function Q to closely approximate the optimal action-value function q_* , independent of the policy in use. This greatly simplifies both the analysis and the proof of convergence for the algorithm. While the policy still influences which state-action pairs are chosen, all state-action pairs must be updated continually for proper convergence [13].

Since we don't have direct access to the true Q^* , we can use a neural network as an approximation tool, given their ability to approximate any function.

A Deep Q-Network (DQN) leverages a neural network to estimate the Q-value function in Q-learning. It employs experience replay, storing previous episodes for off-policy learning. To stabilize training, a target network is used, which is frozen and updated periodically with the latest weights. This helps prevent training instability caused by rapid target updates. By mitigating issues like autocorrelation in online learning, this strategy makes the problem resemble a supervised learning task more closely [14].

2.3 Actor-Critic Methods

Actor-Critic methods aim to combine the advantages of both value-based and policy-based approaches while minimizing their drawbacks. The core idea is to split the model (agent) into two components: one for selecting the best action in a given situation and the other for generating the Q -values of the action or environment. The actor takes the state as input and returns the best possible action based on it. It regulates the agent's behavior by learning an optimal policy (policy-based). On the other hand, the critic computes a value function to evaluate the actions. These two sub-models interact and improve over time. As a result, the overall architecture learns to play the game more efficiently than the two methods would separately. This interaction (or competition) between the two models has

gained significant attention in recent years within the field of machine learning [15].

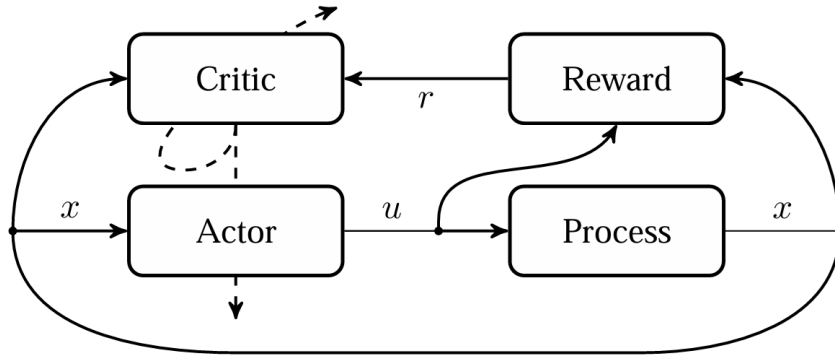


Figure 2.2: Interaction of actor-critic model and environment [16].

The actor-critic method can be imagined as a little boy and his mother. The child (actor) is always trying new things and observing the surroundings. He bumps his head against the wall, touches a hot oven, and eats his own toys. His mother, the critic, watches over him, either praising or criticizing his actions. The child listens to his mother's feedback and adjusts his behavior accordingly. As the child grows, he learns what is right and wrong, and ultimately how to navigate the game of life. This is exactly how the actor-critic method works [15].

The actor's role is to determine the best action for a given state. It is a function approximator, typically a neural network, that performs this task. This could be a convolutional neural network, a fully connected network, or any other architecture. The critic is another function approximator that takes the environment and the actor's action as inputs, combines them, and then outputs an action value (Q -value) for the given state-action pair. To refresh your memory, the Q -value represents the highest possible future reward [15].

The two networks are trained separately using gradient ascent, which updates their weights by finding the global maximum instead of the minimum. Over time, both the actor and the critic improve their ability to make better decisions (the

actor starts to learn the rules). Unlike policy gradients, it is important to note that weights are updated at each step (TD Learning) rather than at the end of the episode [15].

The update rule for the actor's parameters is given by:

$$\theta_{t+1} \doteq \theta_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \theta_t)}{\pi(A_t | S_t, \theta_t)} \quad (2.14)$$

Where θ represents the parameters of the actor's policy, G_t is the return at time step t , and $\pi(A_t | S_t, \theta_t)$ is the probability of taking action A_t given state S_t and policy parameters θ_t [11].

2.4 Asynchronous Advantage Actor Critic

Asynchronous Advantage Actor-Critic (A3C) is a policy gradient algorithm that maintains a policy $\pi(a_t | s_t; \theta)$ and an estimate of the value function $V(s_t; \theta_v)$. It operates in the forward view and uses a mix of n -step returns to update both the policy and the value function. The policy and the value function are updated after every $t_{\max}$ actions or when a terminal state is reached. The update performed by the algorithm can be seen as

$$\nabla_{\theta'} \log \pi(a_t | s_t; \theta) A(s_t, a_t; \theta, \theta_v)$$

which is an estimate of the advantage function given by:

$$\sum_{i=0}^{k-1} \gamma^i r_{t+1} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v)$$

where k can vary from state to state and is upper-bounded by $t_{\max}$. $A(s_t, a_t; \theta, \theta_v)$ is the advantage function, which estimates how much better an action a_t is compared to the average value of state s_t . It is computed as the difference between the

n -step return and the baseline value $V(s_t; \theta_v)$ [17].

The critics in A3C learn the value function while multiple actors are trained in parallel and get synced with global parameters every so often. The gradients are accumulated as part of training for stability - this is like parallelized stochastic gradient descent.

Note that while the parameters θ of the policy and θ_v of the value function are shown as being separate for generality, some of the parameters are always shared in practice. Typically, a convolutional neural network is used that has one softmax output for the policy $\pi(a_t|s_t; \theta)$ and one linear output for the value function $V(s_t; \theta_v)$, with all non-output layers shared [17].

2.5 PPO

A recent development in reinforcement learning called Proximal Policy Optimization (PPO) offers an improvement over Trust Region Policy Optimization (TRPO). This algorithm was proposed in 2017 and showed remarkable performance when it was implemented by OpenAI.

PPO tries to calculate an update at each step that minimizes the cost function while making sure that the deviation from the previous policy is limited. Important criteria are also considered, including ease of implementation, ease of debugging, sample complexity, and sample efficiency. PPO is a policy gradient method that also learns from online data. To ensure minimal deviation in training, it ensures that the updated policy is not drastically different from the old policy. The actor-critic model is the most typical way of implementing PPO [18].

The mathematical equation of PPO is

$$L^{CLIP}(\theta) = \hat{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right], \quad (2.15)$$

where θ is the policy parameter, $\hat{E}_t$ denotes the empirical expectation over time steps, r_t denotes the ratio of the probabilities under the new and old policies respectively, $\hat{A}_t$ is the estimated advantage at time t , and ε is a hyperparameter (usually 0.1 or 0.2).

3. Related Work

Research in action recognition in football videos has mainly concentrated on detecting goals, predefined actions, and video highlights. However, no existing study specifically addresses the recognition of scoring opportunities. Several works lay the groundwork for comparison, demonstrating the challenges and methodologies in related areas. Below, we review three key studies pertinent to this research.

Some of the studies use a mean average precision (mAP) metric to evaluate their models. It is defined as:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (3.1)$$

where AP_i is the average precision for the i -th class, and N is the total number of classes.

3.1 SoccerNet-v2: A Dataset and Benchmarks for Holistic Understanding of Broadcast Soccer Videos

Deliège et al. introduced SoccerNet-v2 [19], a comprehensive dataset designed for action spotting and understanding in broadcast football videos. The dataset contains annotated actions, including goals, substitutions, and card events, providing a benchmark for evaluating models on action recognition tasks. The authors

proposed a two-stage pipeline for action spotting that involves extracting spatiotemporal features from video frames followed by the temporal localization of actions.

The authors organize an annual competition in action spotting. They present the methods submitted to this competition, re-implementing them where necessary to ensure consistent evaluation. Among these methods, the CALF method (Context Aware Loss Function) [16] achieved the best results, with a mean average precision (mAP) of 40.7% on the action spotting task and 72.2% on goal recognition.

This network handles 2-minute chunks of ResNet features and is composed of a spatio-temporal features extractor, kept as is, a temporal segmentation module, which they adapt for 17 classes, and an action spotting module, adapted to output at most 15 predictions per chunk, classified in 17 classes of football actions. The segmentation module is trained with a context-aware loss having four context slicing hyper parameters per class. They determine optimal values for them with a Bayesian optimization [19].

3.2 Automatic Highlights Generation for Sports Videos

Using Audio-Visual Cues

Darapaneni et al. [20] focused on the automatic detection of key events in soccer matches to create video highlights using computer vision. They employed Faster R-CNN [21] and YOLOv5 [22] architectures to identify significant moments, such as goals, corner kicks, and penalties. Their experiments showed that Faster R-CNN with ResNet50 achieved a classification accuracy of 95.5% (they use only 4 different classes), outperforming YOLOv5 and Faster R-CNN with VGG16. The model effectively condensed a 23-minute video to 4 minutes and 50 seconds while capturing nearly all critical match events [20].

The authors created a training set of images consisting of each of the key events (goal, corner, foul and penalty). Their training dataset had only 231 images. They also applied a horizontal flip as data augmentation. They used object detection methodology to identify the important events in a football match. Primarily they used Faster RCNN with VGG16 and ResNet50 pretrained on ImageNet as base models [20].

3.3 Event Detection in Football using Graph Convolutional Networks

Rana [23] explored the application of Graph Convolutional Networks (GCNs) for automatic event detection in football videos. By modeling players and the ball as a graph in each frame, the study demonstrated how GCNs could capture spatial and temporal contexts to detect events. This approach addresses challenges in modeling unstructured tracking data and offers a framework for detecting complex events in sports analytics.

The author utilizes the extracted 2D positions of players and the ball. Those are represented in each frame as graphs that are then fed into a Graph Neural Network architecture to extract features. These features are then pooled, taking into account temporal windows and the pooled features are fed into a multi-label classifier [23].

They also tested their solution on an action spotting task with 12 different classes. Goal chance was also one of the classes. The author achieved different results using different pooling methods. The best results were achieved by methods using versions of VLAD (Vector of Locally Aggregated Descriptors). [24] The best results overall on action spotting task were 31.3% with NetVLAD++. NetRVLAD++ achieved the best results in goal recognition - 34.5%. The best

results for goal chances were achieved by NetVLAD, 32.6%. However, it should be added that NetVLAD scored only 8.5% in goal recognition.

3.4 Summary of Related Work

The reviewed studies demonstrate a variety of methodologies for analyzing football videos, with a primary focus on event detection and action recognition. SoccerNet-v2 provides a benchmark dataset and evaluates models on tasks such as goal recognition and action spotting, with the CALF method achieving state-of-the-art results. Darapaneni et al.. utilized object detection techniques to create video highlights by identifying key events. Meanwhile, Rana investigated the use of Graph Convolutional Networks for event detection, yielding promising results in recognizing goals and spotting actions.

While these works address related problems, none tackle the recognition of scoring opportunities as their main problem. This study differentiates itself by focusing on this unique and underexplored aspect, leveraging reinforcement learning training without the need of a large training dataset.

4. Our Approach

4.1 Introduction to Our Problem

This section defines the problem we aim to address and outlines our approach. Football, the world's most popular sport, has increasingly become a subject of research within computer vision due to its global reach and rich visual data. Every day, countless matches are played worldwide, often exceeding the standard 90-minute duration. Yet, only a small number of significant events occur in each game. To efficiently manage and analyze match recordings, there is a growing need for systems capable of understanding match content and automatically identifying key moments.

Events such as goals, red cards, and penalties are relatively well-defined and, in many cases, easier to detect. However, others—like goal-scoring opportunities (also referred to as chances)—are inherently more ambiguous, lacking a strict definition. This makes them particularly challenging to identify without the aid of automated systems.

The automatic recognition of scoring opportunities can be valuable from several perspectives. It may assist football analysts in evaluating key moments, support coaches in tactical preparation, and enhance the experience for television commentators and fans by providing a clear overview of the most exciting parts of the game.

In our research, we investigate the use of computer vision techniques in combination with reinforcement learning for the purpose of football video analysis. We explore methods that reduce the reliance on large annotated datasets. Instead, our approach employs reinforcement learning, enabling the system to learn through interactions within a simulated football environment.

The primary objective of this work is to design and implement a system capable of detecting goal-scoring opportunities in football footage. This task can be formulated as a binary classification problem: distinguishing between scoring chances and non-chances based on visual input from match videos. These videos are processed as fixed-length sequences that capture short time intervals around moments in the game.

4.2 Solution Design

This section describes our method for detecting goals and goal-scoring opportunities in football match videos by integrating reinforcement learning with computer vision techniques. It thus performs binary classification of goal-scoring chances. Figure 4.1 presents the overall architecture.

On the left side of the diagram, we show the agent’s training process using the Google Research Football environment. This environment is designed for controlled and customizable reinforcement learning experiments. Since we ultimately want to analyze real football videos, we train the agent on visual input, making the feature extractor a critical component for processing raw pixel data.

We employ the Proximal Policy Optimization (PPO) algorithm, enabling the agent to improve through environmental interaction. The agent learns strategies to win (via the actor, which selects actions) and evaluates state quality (via the critic, which assigns a value to each state). States with higher values should correspond

to more advantageous situations—such as goal-scoring chances.

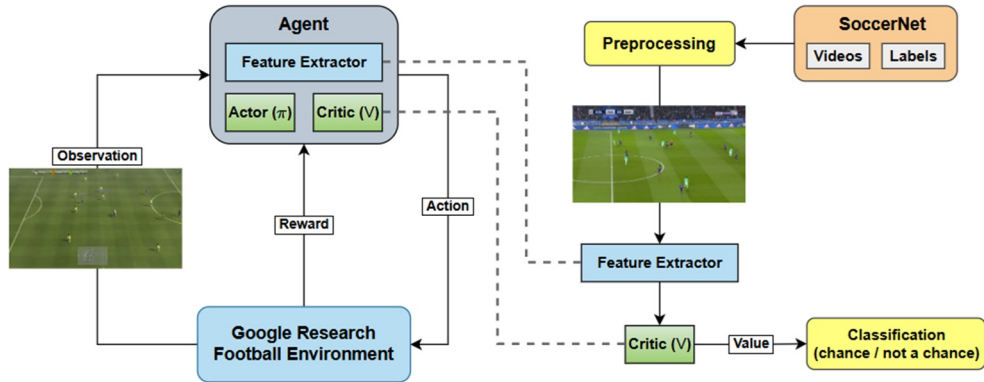


Figure 4.1: Solution Diagram

Once training is complete, we make use of the critic component of the agent. This critic becomes central to our method, as it serves as a proxy for identifying key moments in football videos. Unlike supervised learning, our method does not require labeled training data but instead relies on the critic’s ability to assess frame significance.

On the right side of the diagram, we show how the trained critic is applied to real football footage. The video is preprocessed (e.g., resolution, frame rate) to match the critic’s input format. Each frame is passed through the critic, which assigns it a value representing its importance. This allows us to transfer the critic’s learned capabilities from simulation to real-world video. The critic’s output values are subsequently processed through thresholding to reveal segments of elevated significance.

To evaluate our system, we use our manually annotated dataset of chances and no-chances, which is described in a later chapter. By comparing the critic’s predictions with these annotations, we can assess how effectively the critic detects high-value events.

4.3 Training Environment

Football presents a particularly challenging domain for reinforcement learning, as it requires balancing low-level player control, learned mid-level behaviors such as passing, and high-level strategic decision-making. To train our reinforcement learning agent, we utilized the Google Research Football Environment (Gfootball) [25]. This environment was selected for its realistic simulation of football matches and its flexibility in configuring observational inputs.

Gfootball is specifically designed to teach artificial intelligence how to play football. It comprises several key components: a highly optimized game engine, a collection of demanding research challenges known as football benchmarks, and the Football Academy, which offers progressively difficult training scenarios. In addition, users can define custom scenarios tailored to specific learning goals [25].

At the heart of the environment is the football engine, an advanced simulation built on a heavily modified version of Gameplay Football. It faithfully reproduces football dynamics, including rules such as goals, fouls, corners, penalty kicks, and offsides. The engine is performance-optimized and capable of running efficiently on standard hardware, with or without GPU acceleration. Its C++ implementation has been enhanced for scalability, enabling up to 25 million environment steps per day on a single six-core machine [25].

Gfootball includes features specifically designed for reinforcement learning research. It supports learning from multiple observation modalities, such as structured semantic representations (e.g., player and ball positions) or directly from raw pixel inputs. Moreover, it is fully compatible with the OpenAI Gym API, facilitating seamless integration into modern reinforcement learning pipelines. Collectively, these features make Gfootball a robust and versatile platform for training football-playing agents.

4.3.1 Adapting the Environment

Since our goal is to detect goals and scoring opportunities from visual input, we train our agent using pixel-based representations of the environment. In this setup, the agent observes the game state as rendered images of football scenes. By default, Gfootball renders observations at a resolution of 1280×720 pixels, but this can be adjusted according to experimental needs. We opted for a resolution of 224×224 pixels, which offers a good trade-off between visual clarity (for players and the ball) and computational efficiency. This resolution also aligns well with the input dimensions expected by our model architecture. For comparison, we also conducted experiments using a smaller resolution of 72×96 pixels.

Although single frames capture the spatial configuration of the field, they do not sufficiently convey temporal dynamics—such as the movement of the ball or players—which are critical for effective decision-making. To address this limitation, we apply observation stacking, where we combine the four most recent frames into a single input tensor. Specifically, the first three channels contain RGB values from the most recent frame, while the next three channels contain grayscale difference images that highlight motion between consecutive frames. This approach enables the model to better perceive temporal changes in the scene. The stacking is implemented via a custom environment wrapper. Before input into the model, the stacked observations are normalized with the model’s mean and standard deviation values.

Gfootball provides several customizable features relevant to reinforcement learning, particularly in the design of reward functions. By default, the environment includes scoring rewards, granting +1 for scoring a goal and -1 for conceding. In addition, checkpoint rewards can be enabled to provide intermediate feedback for desirable behaviors, such as successful passes or moving the ball toward the

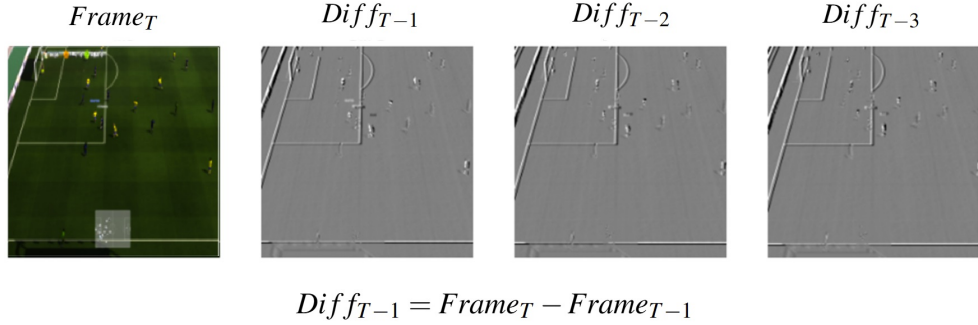


Figure 4.2: Visualization of an observation from the environment. The input has shape (224, 224, 6).

opponent’s goal.

We further extended the environment’s functionality via our custom wrapper. Our modification restricts training episodes to the first half of matches. This simplification ensures that the agent always plays in a consistent direction, which reduces the complexity of the learning problem and helps the agent more quickly develop an understanding of which goal to attack and which to defend.

4.4 Agent Model Architecture

As illustrated in Figure 4.3, our agent model comprises a feature extractor that processes visual inputs to extract meaningful features. This extractor is connected to two heads: one for the actor, which is responsible for selecting actions, and the other for the critic, which evaluates the state of the environment. These heads operate on the flattened output of the feature extractor.

For early-stage experiments and initial debugging, we utilized a ResNet-18 model due to its simplicity, speed, and low memory usage. Then, we transitioned to a more powerful vision transformer model, Eva-02 [26]. This model was selected for its state-of-the-art performance on ImageNet and its ability to capture

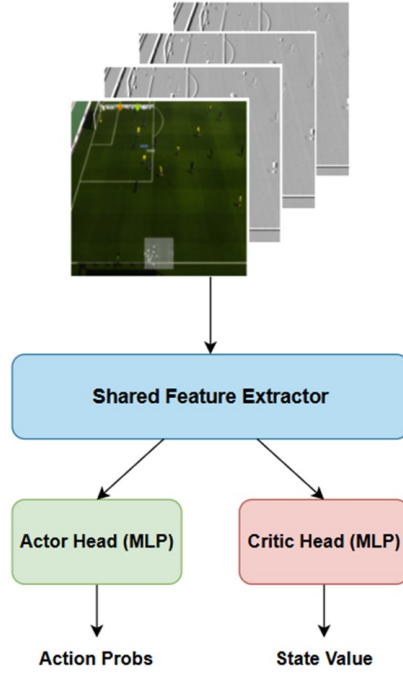


Figure 4.3: Agent model

complex visual patterns, making it suitable for understanding the dynamics of football from pixel-based observations.

In the case of the Eva-02 Vision Transformer, we froze all pretrained parameters except for its input layer (patch embedding layer), which was replaced with a custom version to handle the (224, 224, 6) input resolution. Only this input layer was trained, allowing us to retain the generalization benefits of large-scale pretraining while adapting the model to our specific input format.

The actor and critic heads were implemented as multilayer perceptrons (MLPs) with configurable architectures. Specifically, the critic head consists of layers with sizes [512, 512, 256, 1], while the actor head has layers with sizes [512, 512, 256, 19]. The actor head outputs a probability distribution over the 19 possible discrete actions defined in the Gfootball environment. This corresponds to the policy $\pi(a | s)$ in reinforcement learning. The critic head estimates the value of

the current state $V(s)$, which is used to guide learning during policy updates.

4.5 Training the Agent

Training a reinforcement learning agent in the Gfootball environment using pixel-based observations and a large model is computationally intensive and requires substantial GPU resources. Our experiments were conducted on a server equipped with an Nvidia RTX 4080 GPU.

Figure 4.4 illustrates the training loop and integration of model components. We used the Proximal Policy Optimization (PPO) algorithm implemented in the Stable Baselines3 library [27]. This library was selected for its robust and well-tested PPO implementation, ease of use, and modular design. It allows fine-grained control over all key parameters, integration of custom architectures, real-time logging, periodic evaluation, and efficient saving/loading of models. Moreover, its compatibility with OpenAI Gym environments ensured smooth integration with Gfootball, enabling controlled experiments across various gameplay settings.

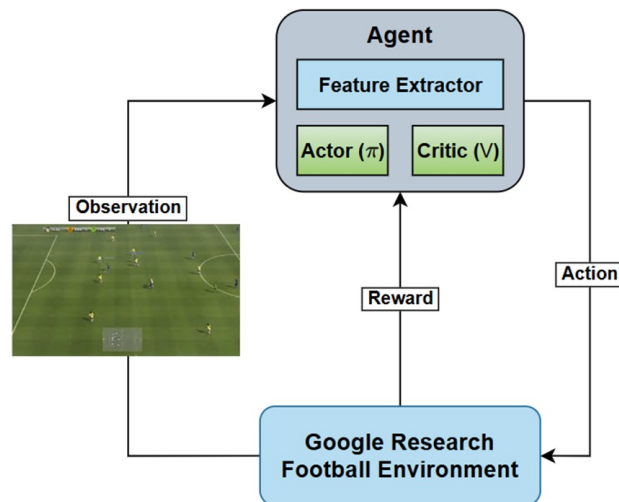


Figure 4.4: Training of reinforcement learning agent

Several hyperparameters were particularly impactful. Among the most important were the number of parallel environments (`n_envs`), number of steps per rollout (`n_steps`), mini-batch size (`batch_size`), discount factor (`gamma`), entropy coefficient (`ent_coef`), number of epochs (`n_epochs`), clipping range, and the learning rate. Based on experience with simpler domains like Atari games, we found that increasing the number of environments generally helps stabilize PPO, and for our setup, we settled on 16 parallel Gfootball environments as a practical balance between stability and GPU constraints. The exact values used in each experiment are presented and discussed in the experimental section.

The result of training is a model composed of two key components: the actor (policy), which selects actions based on current observations, and the critic (value function), which estimates the expected return from a given state.

To evaluate the actor, we measured the average return per match in a separate test environment without exploration noise. In addition to this quantitative evaluation, we qualitatively analyzed gameplay behavior by watching the agent play full matches. To assess the critic, we examined how its value estimates corresponded to key game moments, comparing them with visual gameplay sequences and manually annotated scenarios. This dual analysis ensured the agent was not only learning to act but also to interpret the state of the game meaningfully.

4.6 Validation Dataset Preparation

To evaluate the critic on real-world football scenarios, we used the SoccerNet dataset—a large-scale benchmark designed for video understanding in football. SoccerNet includes 550 full broadcast matches and 12 single-camera games from top-tier European leagues and supports tasks such as action spotting, player tracking, and camera calibration. The dataset is accessible via a Python package,

providing straightforward access to video files in .mkv format and corresponding annotations in .json. Each annotation includes the event type (e.g., goal, foul, corner) and its timestamp within the match. Notably, the video resolution of 1280×720 pixels matches the resolution used in our Gfootball experiments, which allowed for seamless visual comparisons.

Since the critic operates on static image inputs rather than raw video streams, we converted the annotated match data into a frame-based format. For this, we extracted individual frames from selected video segments and applied the same preprocessing pipeline used for Gfootball observations, ensuring visual consistency between domains.

Rather than using entire matches, we focused on specific high-impact events relevant to value estimation. Using the annotations, we extracted frame sequences surrounding key events, sampling 200 frames (8 seconds) around each timestamp (150 frames before annotation time and 50 frames after). These were saved in .png format and organized into directories by event type.

Our validation dataset emphasizes two categories. Positive examples: goals and shots on target, which represent situations that should yield high value estimates from the critic. Negative examples: kick-offs and substitutions, which are not directly related to scoring chances and should produce neutral or low values.

This structure allowed us to assess whether the critic assigns high value to genuine scoring opportunities while remaining unaffected by irrelevant or low-impact events.

4.6.1 Supplementary Manually-Annotated Dataset

The annotations within SoccerNet are useful for our problem, but they are insufficient as they do not follow the format of chance/non-chance. Therefore, to improve the critic’s evaluation, we created a manually annotated dataset focused

on distinguishing scoring chances from situations that do not represent a threat or result in a goal. This additional dataset helps assess whether the critic can differentiate between valuable situations and visually similar but strategically irrelevant moments.

We manually analyzed over 40 full matches from European leagues, sourced from the SoccerNet dataset. We identified 300 chances — moments that resembled goals but did not result in one (such as off-target shots, blocked attempts, or saves) — and 300 non-chances — moments that did not represent realistic scoring opportunities, such as passes in the midfield or other random situations. For statistical purposes and a deeper evaluation of the critic, we also included additional information in the annotations, such as which half of the field the event occurred on, whether the chance was a truly high-value one or not, and in the case of non-chances, whether it contained any potentially misleading situation, such as a pass back to the goalkeeper.

Each moment was extracted in the same way as for SoccerNet, with 200 frames around the specific event timestamp, and saved as .png images in a directory structure. By combining our custom dataset with annotated events from SoccerNet, we created a comprehensive validation framework capable of evaluating the critic’s understanding of explicit and implicit value signals in football gameplay.

4.7 Using Real-World Videos to Validate the Critic

In this section, we describe how real-world football video frames are preprocessed in order to be compatible with the critic model. We also present the evaluation strategy used to assess the critic’s performance on these real-world actions.

4.7.1 Preparing Real-World Observations

To evaluate the effectiveness of the critic on real football footage, we used the manually annotated dataset of chances and non-chances described earlier, based on SoccerNet videos. In order to feed these individual actions into the critic, they first needed to be preprocessed.

The first intuitive step was resizing each frame to 224×224 pixels. Another crucial aspect—especially due to the use of difference frames—is the frame rate. The football simulator runs at 10 FPS, while real-world football videos are recorded at 25 FPS. Therefore, when processing real-world videos, we sampled either every second or every third frame (these approaches are compared in the experimental section). An example of the resulting observation can be seen in Figure 4.5.

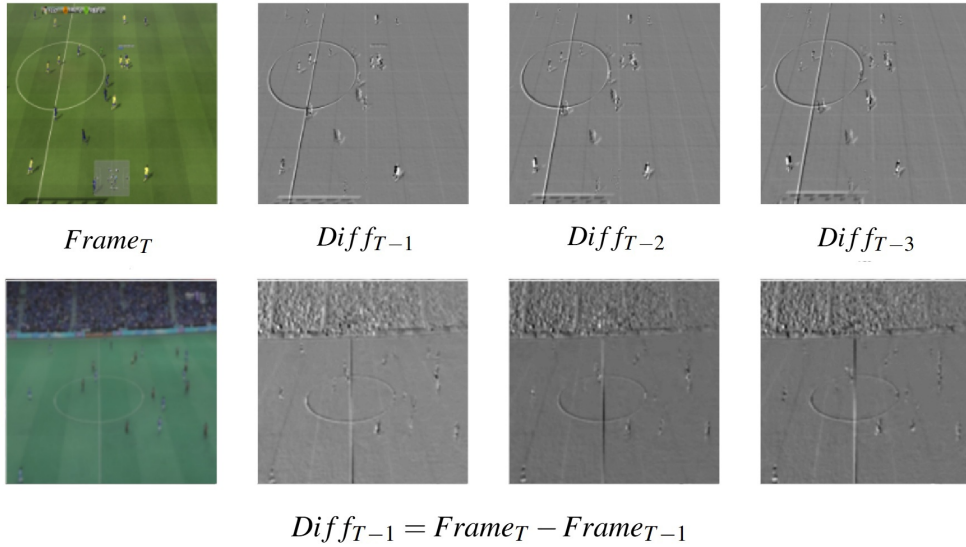


Figure 4.5: Comparison of frames from the gfootball environment (top row) and real football videos (bottom row), used as inputs for the model.

Another important step involved augmenting the frames to minimize the visual discrepancy between real-world videos and observations from the gfootball

environment. For this, we used the Structural Similarity Index Measure (SSIM), which compares images based on luminance, contrast, and structure. SSIM returns a score between -1 and 1, where 1 indicates perfect similarity.

The best-performing augmentation settings, as measured by SSIM similarity to the original video, involved reduced saturation (0.6), lowered contrast (0.6), slight brightness adjustment, and Gaussian blur with a 5×5 kernel (experimentally, a 9×9 kernel yielded slightly better results). We applied a set of augmentations to the frames: saturation, brightness, contrast adjustments, Gaussian blur, and added noise. Each augmentation had several candidate values (i.e., parameters), and a grid search was used to identify the best combination, resulting in an SSIM score of 0.677.

4.7.2 Evaluation of the Critic

The critic returns a sequence of scalar values for each action (i.e., sequence of frames), which is then used to determine whether the action represents a chance or not. Figure 4.6 shows average sequences of chances and no-chances. We first apply aggregation over the sequence of values, using the methods in Table 4.1.

All evaluation metrics are computed from the absolute values of the critic’s outputs, since due to the simulator setup, chances on the left side of the pitch may produce negative values. The aggregation methods include the maximum value (`max_value`), the average of the top-n values (`top_n_avg`), and the range between the minimum and maximum values (`min_max_diff`), as chances are expected to yield higher and more distinct values than non-chances. To capture dynamic patterns, we compute the maximum of the first and second derivatives (`max_derivative`, `max_second_derivative`), as chances are typically accompanied by sharp increases and decreases in the value signal. These are further extended using moving-window variants (`max_moving_value`, `max_moving_derivative`,

Aggregation Method	Mathematical Definition
max_value	$\max_t x_t$
top_n_avg	$\frac{1}{n} \sum_{t \in \text{Top-}n(x)} x_t$
min_max_diff	$\max_t x_t - \min_t x_t$
max_derivative	$\max_t x_{t+1} - x_t $
max_second_derivative	$\max_t x_{t+2} - 2x_{t+1} + x_t $
max_moving_value	$\max_t (\max_{t \leq i < t+w} x_i)$
max_moving_derivative	$\max_t (\max_{t \leq i < t+w-1} x_{i+1} - x_i)$
max_second_moving_derivative	$\max_t (\max_{t \leq i < t+w-2} x_{i+2} - 2x_{i+1} + x_i)$
l1_distance	$\sum_t x_t - \mu_t^{\text{no-chance}} - \sum_t x_t - \mu_t^{\text{chance}} $
l2_distance	$\ x - \mu^{\text{no-chance}}\ _2 - \ x - \mu^{\text{chance}}\ _2$

Table 4.1: Mathematically defined aggregation methods used to compress sequences of critic values into scalar scores. $\text{Top-}n(x)$ denotes the indices of the n largest values in the sequence x . μ_c represents the average value sequence for class c (*chance* or *no-chance*). L1 and L2 distances are computed as the difference between the distance to the *no-chance* average and the distance to the *chance* average. Aggregations with moving use a sliding window of size $w = 5$.

max_second_moving_derivative) to emphasize local trends while reducing the influence of outliers. Finally, to evaluate the overall similarity of an action sequence to typical chance or non-chance patterns, we compute L1 and L2 distances (l1_distance, l2_distance) to the corresponding average value sequences.

Finally, we determine an optimal threshold that best separates chances from non-chances. This threshold is chosen such that the number of false positives and false negatives is balanced (or adjusted according to specific requirements). We then compute standard evaluation metrics: accuracy, precision, recall, and F1 score.

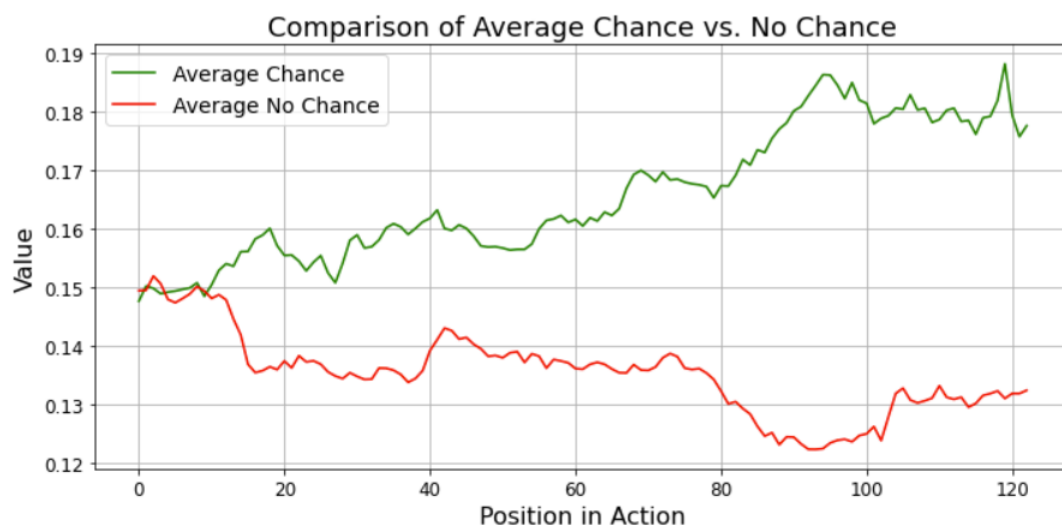


Figure 4.6: Average predicted value across time steps for goal opportunities vs. non-opportunities using the model from Experiment 5.3.

5. Experiments and Results

In this chapter, we describe the individual experiments conducted during this work and evaluate their effectiveness.

5.1 Evaluating the Critic on the Simulator

As described earlier, the first part of our solution involves training an agent within a football simulator. Therefore, verifying whether the critic has learned to evaluate situations properly within the simulation is useful. However, we do not have access to a labeled dataset of chances and non-chances from the simulator. For this reason, the main way to assess the critic during training was by visually inspecting recordings from simulations and monitoring the evolution of predicted values. From this, the critic appeared to be generally effective.

We can partially verify the critic's performance using data. Since goals are inherently great chances, we expect the predicted values around them to be high. Goals can be detected from the environment through reward signals. Therefore, we recorded value sequences around 1000 goals and analyzed them. Figure 5.1 shows the average evolution of predicted values around goals scored on the left and right sides of the pitch.

On average, the critic was able to identify goals with reasonable accuracy. Additionally, values of goals on the left side peaked closer to zero than those on

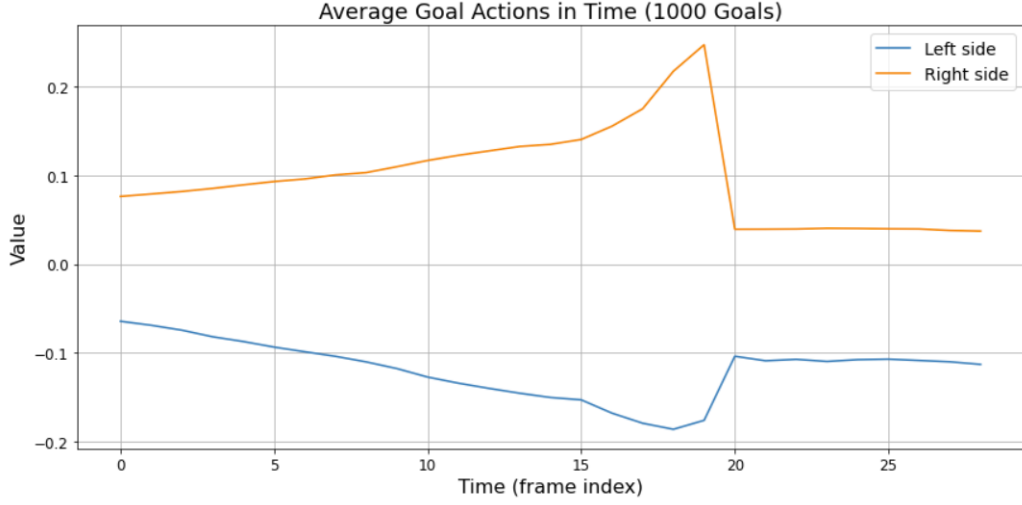


Figure 5.1: Average Chance and No-chance in Gfootball

the right. This is likely because the agent is never perfectly equal to its opponent — it either concedes or scores more goals. In this case, we used a model from Experiment 5.3, where decisions were made by the built-in AI opponent, which outperformed the agent. As a result, the critic encountered more goals on the right side and learned to recognize them slightly better. Since the critic sees different amounts of goals on each side, we naturally expect different performance depending on the direction of play.

For comparison, if we apply the threshold from the corresponding real-world evaluation experiment (based on the maximum second derivative) with $t = 0.081$, we achieve an accuracy of **85.76%** (76.67% on the left side and 90.89% on the right side). However, since real-world videos may generate value sequences within a different range than the simulator, these values are considered only informative.

We applied the threshold selected in real-world evaluation experiment 5.3 (based on the maximum second derivative of the predicted value function), which resulted in an accuracy of **85.76%** (76.67% on the left side and 90.89% on the right side).

Parameter	Value
batch_size	64
n_epochs	2
lr	0.00011879
num_envs	16
n_steps	256
gamma	0.997
ent_coef	0.00155
clip_range	0.115
clip_range_vf	0.115
max_grad_norm	0.76
vf_coef	0.5

Table 5.1: Hyperparameter Configuration of Experiments.

5.2 Classic Actor-Critic PPO Approach

In this experiment, we trained a GFootball agent using the PPO algorithm with hyperparameters listed in Table 5.1. These hyperparameters are inspired by those recommended by the authors of the original GFootball paper. We trained the agent for 75 million steps (14 days).

At the beginning, we conducted 4 shorter experiments with different entropy coefficients (which influence exploration). Based on these results, we initially trained with a higher value ($\text{ent_coef} = 0.01$) to encourage the agent to try more diverse actions. Later, we used the ent_coef specified in the table.

In the second half of training, we noticed that the agent was able to get close to the opponent’s goal but was not shooting, meaning the critic wasn’t seeing enough goals. To address this, we trained the model for 15 million steps on simple

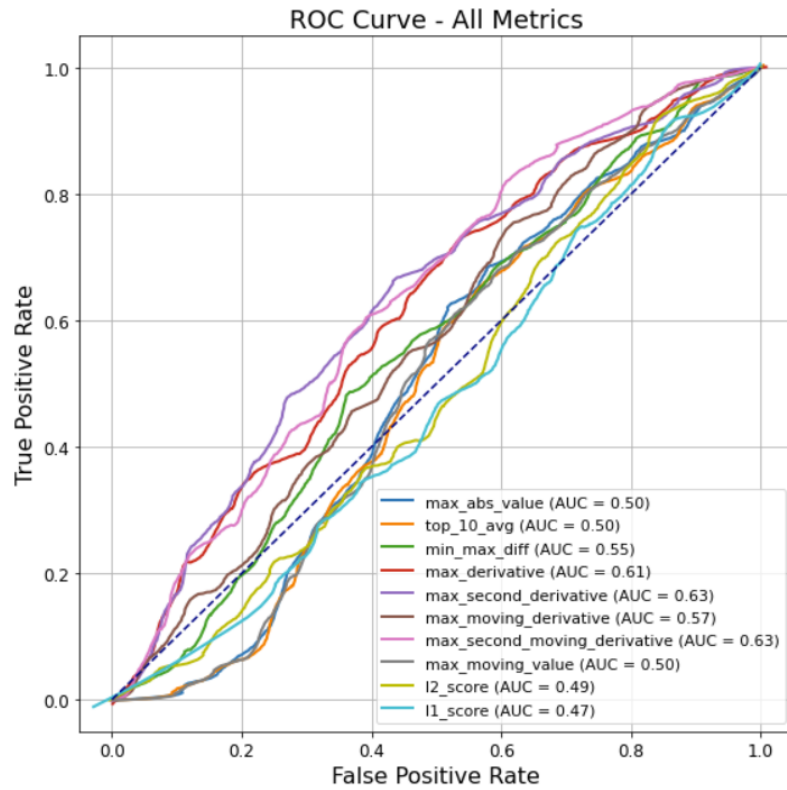


Figure 5.2: ROC curve illustrating the performance of the classic actor-critic experiment, showing the trade-off between the true positive rate and false positive rate at various classification thresholds.

scenarios such as a player alone in front of the goal. This led to better shooting behavior.

Figure 5.2 also shows the ROC curve, which illustrates the trade-off between the true positive rate and false positive rate at various thresholds. Figure 5.3 shows the graph for the max second moving derivative approach, which achieved the best performance. The critic achieved accuracy = **60.73%**, precision = **60.67%**, recall = **60.67%**, and F1 score = **60.67%** on our chances dataset.

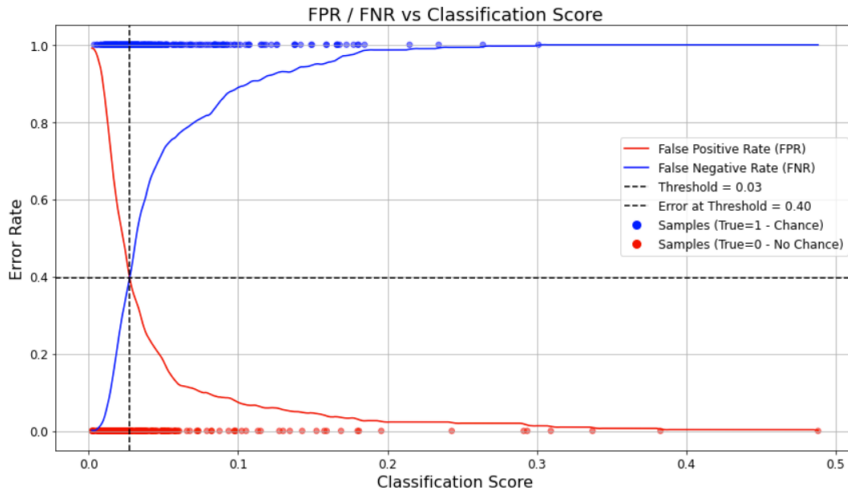


Figure 5.3: Threshold analysis plot for the classic actor-critic experiment, displaying the error rate (FPR and FNR) at various thresholds. The point of intersection represents the optimal threshold selection.

5.3 Only-AI Critic Approach

The problem with the previous experiment was that it took too long for the agent to learn how to score goals. Since ensuring the critic sees enough goals is crucial for us, in this experiment, we continued training the model, but stopped training and using the actor since we're only interested in the critic.

Instead of the actor, we hardcoded action 19 in the GFootball environment, which uses the built-in GFootball AI for all actions. This ensured that the critic saw more goals, and we trained only the critic using critic loss (a deviation from classic PPO). Specifically, instead of the original loss function, which is the sum of `policy_loss`, `entropy_loss` (weighted by the entropy coefficient), and `value_loss` (weighted by the value function coefficient), we simplified the loss to just the `value_loss`.

We trained the agent this way for 100 million steps (18 days). Figure 5.4 also shows the ROC curve, which illustrates the trade-off between the true positive rate

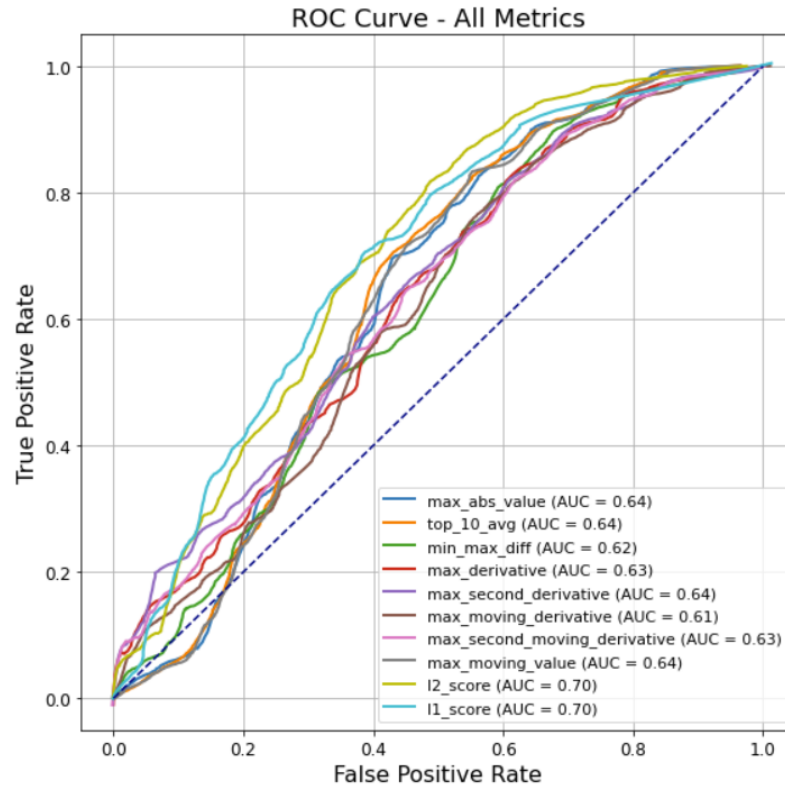


Figure 5.4: ROC curve illustrating the performance of the only-AI critic experiment, showing the trade-off between the true positive rate and false positive rate at various classification thresholds.

and false positive rate at various thresholds. Figure 5.5 shows the graph for the L1 distance approach, which achieved the best performance. The critic achieved accuracy = **65.39%**, precision = **65.33%**, recall = **65.33%**, and F1 score = **65.33%** on our chances dataset.

5.4 Only-AI Critic with Lower Discount Factor

While watching simulation videos and reviewing critic values from the previous experiment, we noticed that state values were significantly above zero, even when the ball was in the centre of the field. This suggests that states leading to goals were

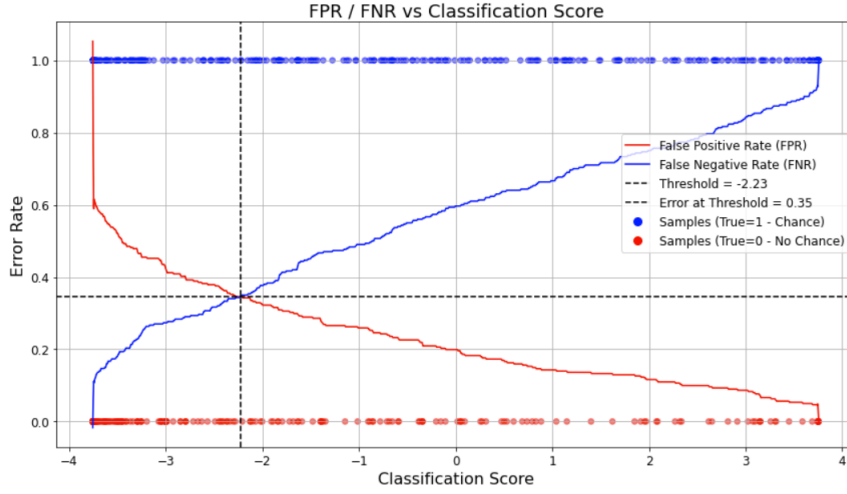


Figure 5.5: Threshold analysis plot for the only-AI critic experiment, displaying the error rate (FPR and FNR) at various thresholds. The point of intersection represents the optimal threshold selection.

overvalued due to a high gamma. Because the agent (now AI-controlled) scored more on the right side, it learned to associate midfield situations with positive outcomes—which is not always true.

In this experiment, we reduced gamma from the original 0.997 to 0.97.

We trained the agent this way for another 65 million steps (11 days). Figure 5.6 also shows the ROC curve, which illustrates the trade-off between the true positive rate and false positive rate at various thresholds. Figure 5.7 shows the graph for the max second derivative approach, which achieved the best performance. The critic achieved accuracy = **62.73%**, precision = **63.04%**, recall = **63.67%**, and F1 score = **61.33%** on our chances dataset.

5.5 Special Cases

As mentioned earlier, the critic detects chances on the right side of the field differently than on the left. This is because there is always some imbalance

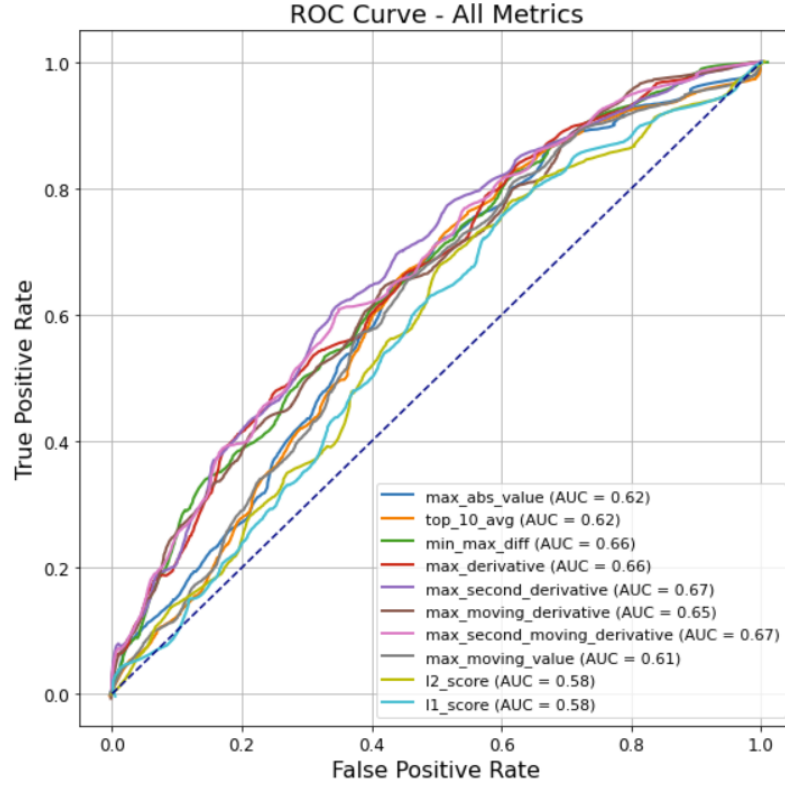


Figure 5.6: ROC curve illustrating the performance of the only-AI critic with lower discount factor experiment, showing the trade-off between the true positive rate and false positive rate at various classification thresholds.

between the two sides during training, which leads the critic to observe different numbers of goals depending on the direction of play—for example, more goals on the right side in our case. Since the critic sees different amounts of goals on each side, we naturally expect different performance depending on the direction of play.

Our dataset of chances and non-chances distinguishes whether they occurred on the left or right. This allows us to evaluate performance separately for each side. We confirmed that performance differs between sides even with real video data (using the best model from experiment 5.3). On the left side, it achieved accuracy = **64.7%**, precision = **67.5%**, recall = **64.6%**, and F1 score = **66.0%**. On the right side, accuracy = **67.5%**, precision = **64.4%**, recall = **67.6%**, and F1

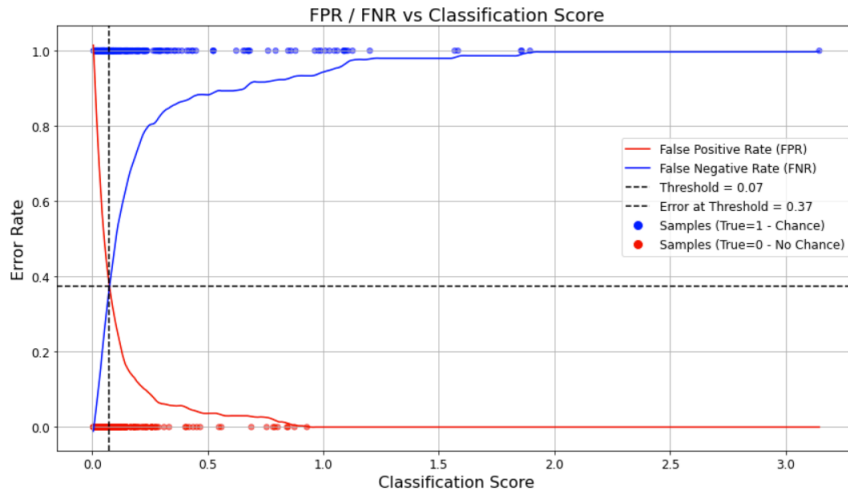


Figure 5.7: Threshold analysis plot for the only-AI critic with lower discount factor experiment, displaying the error rate (FPR and FNR) at various thresholds. The point of intersection represents the optimal threshold selection.

score = **66.0%**.

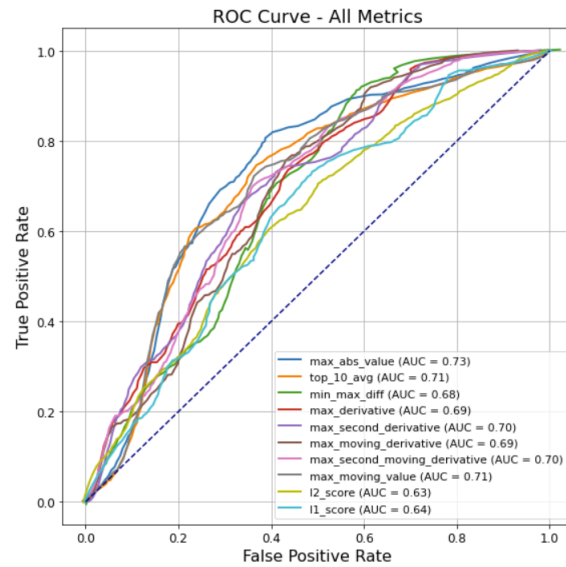


Figure 5.8: ROC curve illustrating the performance of the special cases experiment, showing the trade-off between the true positive rate and false positive rate at various classification thresholds.

In our dataset, for the non-chances, we also annotated whether the situation

might confuse the critic—such as passes to the goalkeeper close to the goal or close-up camera shots of players. We removed these actions from the dataset along with an equal number of chances.

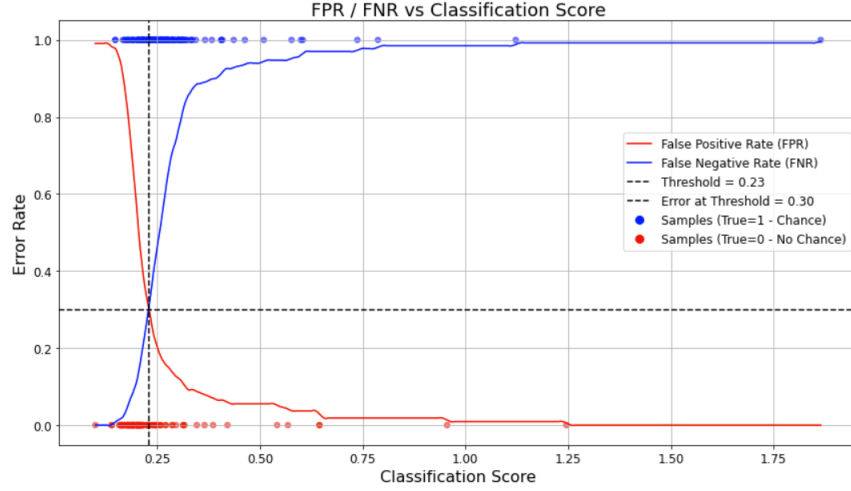


Figure 5.9: Threshold analysis plot for the special cases experiment, displaying the error rate (FPR and FNR) at various thresholds. The point of intersection represents the optimal threshold selection.

Figure 5.8 shows the ROC curve. Figure 5.9 shows the critic’s performance on the right side of the field without these potentially misleading actions. Using maximal value aggregation, we achieved accuracy = **69.71%**, precision = **74.19%**, recall = **69.17%**, and F1 score = **71.60%**.

The overall results of all experiments are summarized in the table 5.2

Experiment	Accuracy	Precision	Recall	F1 score
Classic Actor-Critic PPO	60.73%	60.67%	60.67%	60.67%
Only-AI Critic	65.40%	65.30%	65.30%	65.30%
Only-AI Critic - Lower gamma	62.73%	63.04%	63.67%	61.33%
Special Cases	69.71%	74.19%	69.17%	71.60%

Table 5.2: Summary of critic evaluation results across experiments.

6. Conclusion

This thesis aimed to analyze computer vision and reinforcement learning methods suitable for defining and recognizing scoring opportunities from football videos, to design a system capable of detecting such opportunities, and to implement it. This approach is novel in that, to the best of our knowledge, no prior work has attempted to utilize reinforcement learning in this particular way.

In the initial chapters, we thoroughly analyzed and described the relevant computer vision and reinforcement learning techniques that were either used in our approach or closely related to it. We also reviewed related work, although no previous study has addressed the binary classification of scoring chances directly.

We proposed a solution that leverages a specific property of actor-critic methods: while the agent learns to play football (the actor learns to make decisions), the critic learns to evaluate individual game states. We trained the agent in a football simulation environment and subsequently attempted to repurpose its critic component for classifying real-world football video clips into chances and non-chances. To enable evaluation, we manually annotated and constructed a custom dataset of such clips.

We conducted several experiments. Within our constraints, we were able to demonstrate that the agent successfully learned to recognize scoring opportunities (and goals) in the simulated environment. The key question was to what extent these learned abilities could be transferred to real-world video data. The results

showed that the critic’s performance on real football footage was significantly lower compared to the simulation. The best-performing model achieved 65.30% precision under standard conditions and 74.19% accuracy under specially tailored conditions (more detailed results are in the experiments chapter). However, it consistently outperformed a random classifier, indicating a partial success and confirming that this approach holds promise for future research.

The primary reason for the lower performance on real football videos lies in their inherent differences from the simulation. First, while the simulator aims for realism, real videos feature different camera angles — typically lower and from a fixed broadcast perspective. Second, the nature of actions within real matches differs from the simulation — the goals observed by the agent during training tend to occur in different ways than in real games. Another significant issue was that real-world footage often includes zoom-ins on individual players, which consistently confused the classifier.

Nevertheless, we have demonstrated that this approach has potential. If a more realistic football simulator becomes available in the future, our method could perform substantially better. Similarly, if football matches were recorded in a manner similar to the simulation — or if a dataset of chances and non-chances were constructed using fixed-angle recordings without zoomed-in segments — the classification performance could improve significantly.

The objectives of this thesis were met, and the work provides a solid foundation for further research in this area.

The full implementation is publicly available on GitHub at <https://github.com/RomanGrom/goal-opportunity-spotting-dp>, while the trained models and dataset are hosted on the university’s internal server at [/mnt/nfs-data/public/xgromr](#). For more details on the structure and usage of these resources, see Appendix B.

Bibliography

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning internal representations by error propagation, parallel distributed processing, explorations in the microstructure of cognition, ed. de rumelhart and j. mcclelland. vol. 1. 1986,” *Biometrika*, vol. 71, no. 599-607, p. 6, 1986.
- [3] J. Terven, D. M. Cordova-Esparza, A. Ramirez-Pedraza, E. A. Chavez-Urbiola, and J. A. Romero-Gonzalez, “Loss functions and metrics in deep learning,” 2024.
- [4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems* (I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds.), vol. 30, Curran Associates, Inc., 2017.

- [7] D. Silver, A. Huang, C. Maddison, A. Guez, L. Sifre, G. Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–489, 01 2016.
- [8] K. O’Shea and R. Nash, “An introduction to convolutional neural networks,” 2015.
- [9] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021.
- [10] S. Dara and P. Tumma, “Feature extraction by using deep learning: A survey,” in *2018 Second international conference on electronics, communication and aerospace technology (ICECA)*, pp. 1795–1801, IEEE, 2018.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [12] C. J. Watkins and P. Dayan, “Q-learning,” *Machine learning*, vol. 8, pp. 279–292, 1992.
- [13] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing atari with deep reinforcement learning,” 2013.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

- [15] V. Konda and J. Tsitsiklis, “Actor-critic algorithms,” *Advances in neural information processing systems*, vol. 12, 1999.
- [16] I. Grondman, L. Busoniu, G. A. D. Lopes, and R. Babuska, “A survey of actor-critic reinforcement learning: Standard and natural policy gradients,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 6, pp. 1291–1307, 2012.
- [17] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” 2016.
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [19] A. Delière, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasrollahi, B. Ghanem, T. B. Moeslund, and M. V. Droogenbroeck, “Soccernet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos,” 2021.
- [20] N. Darapaneni, P. Kumar, N. Malhotra, V. Sundaramurthy, A. Thakur, S. Chauhan, K. Thangeda, and A. Paduri, “Detecting key soccer match events to create highlights using computer vision,” 04 2022.
- [21] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” 2016.
- [22] M. Hussain, “Yolov5, yolov8 and yolov10: The go-to detectors for real-time vision,” 2024.
- [23] A. S. S. Rana, “Event detection in football using graph convolutional networks,” 2023.

- [24] H. Jégou, M. Douze, C. Schmid, and P. Pérez, “Aggregating local descriptors into a compact image representation,” in *2010 IEEE computer society conference on computer vision and pattern recognition*, pp. 3304–3311, IEEE, 2010.
- [25] K. Kurach and O. Bachem, “Introducing google research football: A novel reinforcement learning environment,” *Google Blog*, 2019.
- [26] Y. Fang, Q. Sun, X. Wang, T. Huang, X. Wang, and Y. Cao, “Eva-02: A visual representation for neon genesis,” *Image and Vision Computing*, vol. 149, p. 105171, Sept. 2024.
- [27] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

Used AI Tools

- OpenAI (2025), ChatGPT, Entire text translation between languages
- OpenAI (2025), ChatGPT, Revisions and reformulations of the entire text
- OpenAI (2025), ChatGPT, Generation of computer programs
- Grammarly (2024), <https://app.grammarly.com>, Grammar correction for the entire text

Appendix A: Resumé v slovenčine

Reg. No.: FIIT-182905-102924

Úvod

Futbal je najpopulárnejší a najsledovanejší šport na svete. Každý deň sa odohráva veľké množstvo zápasov po celom svete, počas ktorých vznikajú rozsiahle videozáznamy. Ručné spracovanie týchto záznamov je časovo náročné, preto vzniká potreba automatizácie.

Zápasy sú dlhé, ale množstvo skutočne zaujímavých momentov (kľúčových momentov) je relatívne malé. Automatická detekcia týchto momentov je užitočná, pretože znižuje potrebu manuálneho prezerania. Niektoré kľúčové momenty, ako napríklad góly, sú jednoznačne definované a priamočiarejšie na rozpoznávanie. Iné, ako príležitosti na skórovanie (šance), sú viac subjektívne a nie sú exaktne definované. Preto si ich rozpoznávanie vyžaduje iný, samostatný prístup.

Cieľom tejto práce je navrhnúť a implementovať systém, ktorý by bol schopný definovať a automaticky rozpoznávať šance z obrazových záznamov futbalu. Práca si kladie za úlohu pri tomto probléme využiť metódy počítačového videnia s použitím učenia posilňovaním (reinforcement learning). V praxi ide o problém binárnej klasifikácie šancí.

Analýza metód v skratke

V tejto časti stručne zhrnieme, opíšeme a vysvetlíme použité metódy z oblasti počítačového videnia a učenia posilňovaním.

Metódy počítačového videnia

Pri spracovaní obrazu zohráva kľúčovú úlohu extrakcia príznakov (feature extraction), ktorej cieľom je transformovať obrazové dáta do kompaktnej a zároveň informatívnej reprezentácie. Tradične sa príznaky extrahovali ručne. Dnes sa využívajú najmä hlboké neurónové siete, ktoré sa tieto reprezentácie učia automaticky.

Klasicky sa na extrakciu príznakov používajú konvolučné neurónové siete, respektíve ich prvé vrstvy. Osobitne účinné sú však Vision Transformery (ViT), ktoré aplikujú mechanizmus self-attention priamo na obrazové dáta rozdelené do menších „patchov“. Každý patch je prevedený na vektor a doplnený o pozičné informácie, pričom ViT zachytáva globálne vzťahy medzi jednotlivými časťami obrazu už v prvých vrstvách siete. Výstupom je množina vektorov, ktorá môže byť ďalej spracovaná klasickou plne prepojenou neurónovou sieťou.

Učenie posilňovaním

Učenie posilňovaním (reinforcement learning) je oblasť strojového učenia, ktorá sa snaží imitovať spôsob ľudského učenia. Agent interaguje s prostredím a snaží sa vyberať čo najlepšie akcie v rôznych situáciách tak, aby maximalizoval numerickú odmenu. Agent pri tréningu skúša rôzne akcie a učí sa, ktoré v akej situácii generujú najvyššie odmeny. Tento proces sa často modeluje ako Markovský rozhodovací proces, kde agent v diskrétnych časových krokoch pozoruje stav prostredia, vykonáva akcie a získava odmenu. Cieľom agenta je maximalizovať súčet budúcich odmien, pričom výber akcie závisí iba od aktuálneho stavu. Kľúčovými komponentmi sú politika (policy), ktorá definuje správanie agenta, a hodnotová funkcia (value function), ktorá odhaduje očakávanú návratnosť z daného stavu.

Jednou z metód učenia posilňovaním je actor-critic metóda, ktorá kombinuje úlohu politiky a hodnotovej funkcie. Actor vyberá akcie podľa aktuálnej politiky, zatiaľ čo critic hodnotí ich kvalitu prostredníctvom odhadu hodnotovej funkcie. Jednou z najúspešnejších implementácií takejto architektúry je algoritmus Proximal Policy Optimization (PPO), ktorý zvyšuje stabilitu tréningu zavedením obmedzenia na zmenu politiky medzi iteráciami učenia.

Existujúce práce

Táto práca sa odlišuje od existujúcich prác tým, že sa zameriava čisto na rozpoznávanie šancí, čomu sa doterajšie štúdie priamo nevenujú. Typicky sa práce venujú rozpoznávaniu viacerých tried akcií z futbalu, ako napríklad pri súťaži „Action Spotting Challenge“, ktorú organizuje SoccerNet. Ide pri nej o rozpoznávanie 17 rôznych tried, ako napríklad góly, penalty alebo červené karty.

Jediná práca, ktorá zahŕňa aj rozpoznávanie šancí, je štúdia (Rana), ktorá využila grafové neurónové siete. Hoci sa primárne zameriava na všeobecnú detekciu rôznych udalostí vo futbalových videách, medzi klasifikovanými triedami bola aj kategória „šanca“. Najlepší výsledok dosiahol model NetVLAD, konkrétne 32,6% mAP (mean average precision). Avšak treba poznamenať, že úspešnosť modelu pri rozpoznávaní gólov bola iba 8,5%.

Návrh riešenia

Ako sme spomínali v úvode, cieľom práce je navrhnúť systém, ktorý dokáže rozpoznávať šance z futbalových videozáznamov a využíva metódy počítačového videnia a učenia posilňovaním. Využili sme na to princíp actor-critic metód. Pri tom, ako sa actor učí robiť rozhodnutia v hre, critic sa učí odhadovať hodnoty jednotlivých stavov hry. Vyššie hodnoty stavov by mali predstavovať výhodné pozície v hre, čo pri futbale v praxi predstavujú šance.

Prvou časťou nášho riešenia je preto tréning agenta učenia posilňovaním na reali-

stickom simulátore futbalu pomocou algoritmu PPO. Pri ňom sa critic naučí rozpoznávať šance v rámci simulátora. Druhou časťou riešenia je použitie critica na reálnych futbalových záznamoch. Na to využijeme predspracovaný dataset futbalových momentov. Na konci vyhodnotíme úspešnosť nášho riešenia a ako veľmi sa podarilo preniesť naučené schopnosti zo simulátora na reálne videá.

Tréningové prostredie

Ako simulátor futbalu sme si vybrali Google Research Football (GFootball). Ide o realistický simulátor futbalu s dôrazom na výskum v oblasti učenia posilňovaním. Jeho engine verne simuluje futbalové pravidlá (góly, fauly, penalty atď.). Je kompatibilný s OpenAI Gym API, čo zjednodušuje prácu s ním. Je flexibilný v prispôbení a podporuje rôzne typy pozorovaní, aj výstup v surových pixeloch v kubovoľnom rozlíšení.

Pre naše potreby sme pozorovania zo simulácie upravili. Jedna snímka z prostredia nestačí na zachytenie pohybu, ktorý je pre porozumenie futbalu kľúčový. Preto používame stacking pozorovaní, pri ktorom sa k poslednému RGB pozorovaniu pridajú tri čiernobiele rozdielové snímky, ktoré v praxi zachytávajú pohyb na posledných 4 snímkach. Pozorovanie zo simulácie má preto 6 kanálov.

Druhou výraznejšou úpravou tréningového prostredia je to, že trénujeme agenta iba na prvých polčasoch hry. Agent preto hrá vždy v jednom smere – bráni naľavo a útočí napravo. Malo by to urýchliť jeho učenie.

Architektúra agenta

Architektúra modelu agenta odzrkadľuje, že ide o actor-critic metódu. Pozostáva z extraktora vizuálnych príznakov a dvoch hláv – jednej pre actora a druhej pre critica.

Extraktor príznakov spracováva obrazové vstupy a výstupy posúva actorovi a criticovi. Použili sme Vision Transformer Eva-02 pre jeho vysokú úspešnosť na ImageNete a schopnosť spracovávať komplexné vizuálne vzory. Zmrazili sme jeho parametre a trénovali iba

jeho prvú vrstvu, ktorú sme nahradili novou, aby bol schopný spracovávať naše 6-kanálové vstupy.

Na neho sme pripojili dve samostatné hlavy – jednu pre politiku (actor) a druhú pre hodnotovú funkciu (critic). Obe hlavy sú plne prepojené neurónové siete, ktoré využívajú výstup z transformera.

Tréning agenta

Na tréning agenta sme použili algoritmus PPO implementovaný v knižnici Stable Baselines 3. Túto knižnicu sme si vybrali pre spoľahlivú implementáciu algoritmu PPO a možnosť nastavovania všetkých parametrov spolu s možnosťou integrácie vlastnej architektúry. Gfootball prostredie bolo ľahko integrovateľné vďaka kompatibiliti s OpenAI Gym API.

Medzi najvýznamnejšie hyperparametre patrili počet paralelných prostredí (kľúčové pre rýchlosť tréningu), dĺžka rolloutov, veľkosť batchu, diskontný faktor (gamma), koeficient entropie, klipovacia hranica alebo learning rate.

Tréning bol výpočtovo náročný, keďže agent sa učil z obrazových vstupov zo simulácie prostredia pomocou veľkého modelu. Prebiehal na GPU Nvidia RTX 4080 na školskom serveri rádovo v jednotkách týždňov.

Validačný dataset

Na to, aby sme mohli použiť natrénovaného critica na reálnych videách potrebujeme validačný dataset. Na to sme využili SoccerNet, ktorý obsahuje cez 500 video záznamov z kompletných futbalových zápasov z európskych líg a anotáciami rôznych tried momentov. Videá sú v .mkv formáte a anotácie v .json.

Na vyhodnotenie úspešnosti critica potrebujeme taký dataset, ktorý bude rozlišovať šance a nešance. SoccerNet sám o sebe rozlišuje triedy, ako góly alebo penalty, ktoré vieme využiť ako šance a úvodné rozohrávky ako nešance. To sme aj využili pri úvodných experimentoch, no nebolo to pre nás dostatočné, keďže náš problém je klasifikácia šancí

a nešancí a nad takýmto datasetom sme nemali dostatočnú kontrolu.

Preto sme sa rozhodli vziať futbalové videá zo SoccerNetu a manuálne anotovať momenty v nich, ktoré boli šancami a ktoré neboli. Takto sme z viac ako 40 celých zápasov vytvorili ručne anotovaný dataset s 300 šancami a 300 nešancami. Anotácie obsahujú informácie aj o strane ihriska, kde sa akcia stala a o type konkrétnej akcie (napríklad či to bola veľká šanca). Dataset je uložený v adresárovej štruktúre, pričom sú snímky uložené ako .png súbory (pre každú akciu 200 snímok).

Použitie reálnych videí

Na to, aby sme reálne videá (náš dataset) vedeli použiť na criticovi a validovať ho, musíme ich najskôr predspracovať. Reálne pozorovania musia byť v rovnakej podobe, ako pozorovania zo simulátora, teda 6-kanálové s rozdielovými snímkami. Musíme myslieť na to, že simulátor beží pri 10fps a reálne videá sú v 25fps. Preto pri vytváraní rozdielových snímok berieme každú druhú alebo každú tretiu snímku.

Druhým dôležitým krokom predspracovania je minimalizovanie vizuálnych rozdielov medzi reálnymi videami a pozorovaniami zo simulátora pomocou augmentácií. Na tento účel sme okrem vizuálneho porovnávania použili index štruktúrálnej podobnosti (SSIM). SSIM porovnáva snímky na základe luminiscencie, kontrastu a štruktúry a vracia skóre medzi -1 a 1, kde 1 znamená úplnú podobnosť. Na zlepšenie podobnosti sme na snímky aplikovali úpravu jasu, saturácie a kontrastu a pridávali Gaussovské rozmazanie a šum. Pomocou grid searchu sme našli najlepšiu kombináciu, pri ktorej bola podobnosť 0.677.

Evaluácia critica

Každá akcia je sekvencia snímok, a preto critic vráti pre každú akciu sekvenciu hodnôt. Z tejto sekvencie sa snažíme klasifikovať, či išlo o šancu, alebo nie. Najskôr využijeme agregáciu hodnôt týchto sekvencií. Tie základné sú priamočiare a ide o maximálnu hodnotu sekvencie, priemernú hodnotu a rozdiel medzi minimálnou a maximálnou hod-

notou. Keďže hodnoty sekvencie šance by mali ku danému momentu stúpať a za ním zase klesnúť, použili sme aj maximálnu prvú deriváciu a maximálnu druhú deriváciu. K týmto vymenovaným (maximálna hodnota, prvá a druhá derivácia) sme ešte pridali ich verzie s pohyblivým oknom veľkosti 5, aby sme spriemerovali susediace hodnoty.

Po vykreslení grafu priemernej šance a priemernej nešance sme videli, že rozdiel medzi nimi rozhodne je. Preto sme pridali k spomínaným agregáciám ešte L1 a L2 vzdialenosť celých sekvencií od priemernej šance a priemernej nešance.

Keďže hodnoty pri šanciach na ľavej strane ihriska by mali byť záporné, vždy sme používali absolútnu hodnotu. Následne sme podľa týchto agregácií nastavovali threshold, ktorý čo najlepšie oddelí množinu šancí a množinu nešancí. Nastavovali sme na hodnotu, kde sa množstvo falošne pozitívnych výsledkov rovnalo množstvu falošne negatívnych.

Experimenty a výsledky

V tejto časti popisujeme jednotlivé experimenty vykonané počas tejto práce a hodnotíme ich úspešnosť.

Hodnotenie critica na simulátore

Pri prvom experimente sme sa snažili overiť, či sa critic dokázal naučiť rozpoznávať šance v rámci simulácie. Keďže nemáme anotovaný dataset šancí a nešancí zo simulácie, hodnotenie critica sme vykonávali najmä vizuálne tak, že sme si pri zázname simulácie zobrazovali vývoj hodnôt. Hodnoty väčšinu šancí odzrkadlili.

Avšak, chceli sme si to overiť aj experimentálne, preto sme zaznamenali sekvencie hodnôt okolo 1000 gôlov a skúmali, či ich critic rozpoznal. Na grafe priemerných hodnôt okolo gôlov pre ľavú a pravú stranu ihriska boli góly jasne viditeľné. Keď sme pre porovnanie použili threshold z experimentu s reálnymi videami (pomocou maximálnej druhej derivácie), dostali sme úspešnosť 85.76% (76.67% na ľavej strane ihriska a 90.89% na pravej). Avšak pre reálne videá môžu byť hodnoty posunuté, preto sú tieto výsledky

skôr len informatívne.

Klasický PPO prístup

V tomto experimente sme trénovali agenta na Gfootballe pomocou PPO algoritmu. Hyperparametre sme použili také, aké odporúčali autori Gfootballu. Tréning trval 75 miliónov krokov (14 dní). Prvé dni sme zvýšili koeficient entropie, aby agent skúšal viac rôznych akcií.

Počas druhej polovice tréningu sme si všimli, že agent sa dokáže dostať k súperovej bráne, no dostatočne na ňu nestriela. Preto sme ho 3 dni trénovali na jednoduchších scenároch okolo súperovej brány. Naučil sa viac strieľať na bránu.

S takto natrénovaným modelom sme na reálnych videách pomocou agregácie cez druhú deriváciu dosiahli accuracy = 61.4%, precision 61.5%, recall: 60.7%, F1 score: 61.1%.

Prístup s AI actorom

Pri predošlom experimente sme si uvedomili, že trvá dlho, kým sa actor naučí hrať dobre a dávať viac gólov (je potrebné, aby critic videl góly). Preto sme miesto nášho nastavili v gfootball prostredí stabilne akciu č. 19, ktorá predstavuje zabudovanú AI, ktorá robí rozhodnutia. Critica sme trénovali ďalej, ako doteraz (iba na základe critic loss). Týmto spôsobom videl oveľa viac gólov.

Takto sme model agenta trénovali 100 miliónov krokov (18 dní). Kritik na reálnych videách pomocou prístupu s L1 vzdialenosťou accuracy = 65.4%, precision = 65.3%, recall = 65.3%, F1 score = 65.3%.

Nižší diskontný faktor

Všimli sme si, že v rámci simulácie majú situácie v strede ihriska vyššiu hodnotu, ako sme čakali (čakali sme niečo blízko 0). Pravdepodobne to bolo spôsobené tým, že zabudovaná

AI, ktorá teraz robí rozhodnutia je lepšia ako súper a preto videl critic viac jeho gólov (s kladnými hodnotami). Preto sme nastavili diskontný faktor gamma na nižšiu hodnotu - miesto 0.997 na 0.97. Malo by to spôsobiť, že odmeny za góly budú kratšie do minulosti ovplyvňovať stavy pred gólom.

Takto sme agenta trénovali ďalších 65 miliónov krokov (11 dní). Hoci hodnoty v strede ihriska sa skutočne priblížili k 0, úspešnosť dosiahol horšiu (pomocou maximálnej druhej derivácie): accuracy = 62.73%, precision = 63.04%, recall = 63.67%, F1 score = 61.33%.

Špeciálne prípady

Pri experimentoch sme skúsili zlepšiť úspešnosť rôznymi prístupmi. Jednou z vecí, ktoré sme si všimli bol fakt, že critic nie je rovnako úspešný na oboch stranách ihriska. Často bol lepší na pravej strane (keďže tam agent videl viac gólov pri tréningu). A ak bol aj podobne dobrý, mal lepšiu úspešnosť, ak klasifikoval zvlášť akcie na pravej strane ihriska a na ľavej.

Druhou vecou, ktorá spôsobila lepšiu úspešnosť bolo odstránenie nešancí, ktoré mohli kritika zmiasť (mali sme takú anotáciu v rámci datasetu - išlo o momenty, ako nahrávka vlastnému brankárovi, alebo že sa vo videu objavilo priblíženie na nejakého hráča). Odstránili sme rovnaký počet šancí. A skutočne táto zmena pomohla zlepšiť úspešnosť.

Po klasifikovaní pravej a ľavej strany zvlášť a zmenšení datasetu sme dosiahli pomocou L1 vzdialenosti accuracy = 69.71%, precision = 74.19%, recall = 69.17%, F1 score = 71.60%.

Zhodnotenie

Cieľom tejto práce bolo preskúmať metódy počítačového videnia a reinforcement learningu s cieľom navrhnúť systém na rozpoznávanie gólových šancí z futbalových videí. Navrhli sme prístup založený na actor-critic architektúre, kde počas tréningu v simulovanom

prostredí actor optimalizuje rozhodovanie a critic sa učí hodnotiť jednotlivé stavy hry. Následne sme critic časť využili ako binárny klasifikátor na identifikáciu šancí v reálnych zápasových záznamoch.

Na podporu experimentov sme vytvorili vlastný dataset obsahujúci anotované situácie so šancami a nešancami. Výsledky ukázali, že model dokáže rozpoznať šance v simulovanom prostredí spoľahlivo, pričom v reálnych videách dosiahol precision 65,30% v štandardnom nastavení a 74,19% v prispôsobenom režime. Napriek poklesu presnosti oproti simulácii výsledky významne prevyšovali náhodnú klasifikáciu, čo indikuje schopnosť modelu generalizovať.

Hlavné obmedzenia vyplývali z rozdielov medzi simuláciou a realitou — najmä odlišného uhla kamery, dynamiky herných situácií a častých priblížení na hráčov. Napriek tomu práca demonštruje potenciál tohto prístupu a predstavuje perspektívny základ pre budúci výskum v oblasti automatického rozpoznávania herných situácií vo futbale.

Appendix B: Opis digitálnej časti

Reg. No.: FIIT-182905-102924

Odovzdaný projekt je tiež verejne dostupný na <https://github.com/RomanGrom/goal-opportunity-spotting-dp>. Nachádzajú sa tam docker/build.sh a docker/run.sh skripty na spustenie docker kontajneru s projektom. Podrobný návod a popis, ako spustiť skripty, ktoré produkujú výsledky uvedené v práci sa nachádza v README.md súbore.

Natrénované modely sa nachádzajú na školskom serveri na /mnt/nfs-data/public/xgromr/models. Taktiež sa tam nachádza dataset šancí - /mnt/nfs-data/public/xgromr/chances_dataset.tar.